

Python Programming - 2301CS404

Lab - 2

01) WAP to print "Hello World..!!"

```
In [1]: print("Hello Word.")
```

Hello Word.

02) WAP to accept your name and display a welcome message.

Input: Priya

Output: Hello Priya, welcome to Python Lab.

```
In [2]: name = input("Enter Your Name : ")  
  
print("Hello " ,name ,", welcome to Python Lab.",sep = "")
```

Hello vi, welcome to Python Lab.

03) WAP to accept three integers and display the numbers, their sum, and average.

Input: 10 20 30

Output:

Numbers: 10 20 30

Sum: 60

Average: 20.0

```
In [8]: num1 = int(input("Enter Number 1 : "))  
num2 = int(input("Enter Number 2: "))  
num3 = int(input("Enter Number 3 : "))  
  
print(num1 , num2 , num3)  
  
print("Sum : " , (num1 + num2 + num3))  
  
print("Avg : ",(num1 + num2 + num3)/3)
```

```
1 3 6
Sum : 10
Avg : 3.3333333333333335
```

04) WAP to accept name (string), age (int), and percentage (float).

Input : Riya,18,92.5

Output :

Name: Riya Type: <class 'str'>

Age: 18 Type: <class 'int'>

Percentage: 92.5 Type: <class 'float'>

```
In [9]: name = input("Enter Name : ")
age = int(input("Enter Age : "))
percentage = float(input("Enter Percentage : "))

print(name , type(name))
print(age , type(age))
print(percentage , type(percentage))
```

```
vi <class 'str'>
19 <class 'int'>
99.99 <class 'float'>
```

05) WAP to print folowing message using custom separator and end.

Ooutput : Python | Programming | Basics###

```
In [10]: print("Python","Programming","Basics",sep=" | ",end = "###")
```

Python | Programming | Basics###

06) WAP to accept a value and display its value, type, and memory id.

Input : hello

Output :

Value: hello

Type: <class 'str'>

ID: 140712345678912

```
In [12]: val = input("Enter Value : ")

print(val)
print(type(val))
print(id(val))
```

```
vi
<class 'str'>
1618009070912
```

07) WAP to assign a value to a variable, print id, reassign a new value, and print id again.

Output :

Original ID of a: 140712345678912

New ID of a: 140712345678960

```
In [13]: val = input("Enter Value : ")

a = val

print(id(a))

a = 7

print(id(a))
```

```
1617992841232
140720062510184
```

08) WAP to print multiple lines using a single print().

Output:

Welcome to Python

This is the second lab

Enjoy coding!

```
In [16]: print("""Welcome to Python
This is the second lab
Enjoy coding! """)
```

```
Welcome to Python
This is the second lab
Enjoy coding!
```

09) WAP to display following table of items with proper alignment.

Output :

Sr No	Name	Subject	Grade	Percentage
1	Nisha Patel	Math	A	92
2	Aarav Modi	Science	B+	85
3	Jiya Shah	English	A+	96

```
In [64]: print(f"{'SrNo':^15}{'Name':15}{'Subject':15}{'Grade':^15}{'Percentage':>15}")
print(f"{1:^15}{'Nisha Patel':15}{'Math':15}{'A':^15}{92:>15}")
print(f"{2:^15}{'Aarav Modi':15}{'Science':15}{'B+':^15}{85:>15}")
print(f"{3:^15}{'Jiya Shah':15}{'English':15}{'A+':^15}{96:>15}")
```

SrNo	Name	Subject	Grade	Percentage
1	Nisha Patel	Math	A	92
2	Aarav Modi	Science	B+	85
3	Jiya Shah	English	A+	96

10) WAP to accept a float number and display with 2 decimals, 3 decimals, and width 10.

Input : 37.2567

Output :

2 decimals: 37.26

3 decimals: 37.257

Width 10: 37.26

```
In [93]: num = float(input("Enter Num : "))

print("2 decimal" , f"{num : .2f}")
print("3 decimal" , f"{num : .3f}")
print("Width 10" , f"{num : 10.2f}")
```

```
2 decimal  1.00
3 decimal  1.000
Width 10    1.00
```

11) WAP to accept two integers and display sum, difference, and product using f-strings.

Input : 12 8

Output :

Sum = 20

Difference = 4

Product = 96

```
In [75]: num1 = int(input("Enter Num1 : "))
num2 = int(input("Enter Num2 : "))
print(f"{num1+num2}")
print(f"{num1-num2}")
print(f"{num1*num2}")
```

```
8
-2
15
```

12) WAP to accept date in dd mm yyyy format and display in multiple formats.

Input : 01 12 2025

Output :

01/12/2025

2025-12-01

```
In [78]: day , month , year = input("Enter Date in dd mm yyyy formate.").split()

print(day + "/" + month + "/" + year)

print(year + "-" + month + "-" + day)
```

07/11/2006

2006-11-07

13) WAP to calculate area and perimeter of a circle.

```
In [ ]: r = int(input("Enter r : "))

print("Area : ", 3.14 * r * r)
print("Area : ", 3.14 * 2 * r)
```

14) WAP to convert degree into Fahrenheit and vice versa.

```
In [94]: c = float(input("Enter Degree : "))

print((9/5) * c + 32) # its convert into fahrenheit

# f = float(input("Enter Fahrenhit : "))

# print((5/9) * (f - 32)) # its convert into Degree
```

77.0

15) WAP to get the distance from user into kilometer, and convert it into meter, feet, inches and centimeter.

```
In [91]: km = int(input("Enter distance (km): "))

m = km * 1000

cm = m * 100

inches = cm / 2.54 # 1 inch = 2.54 cm

ft = inches / 12 # 1 foot = 12 inches

print("Meters      :", m)
print("Centimeters:", cm)
print("Inches      :", inches)
print("Feet        :", ft)
```

Meters : 25000
Centimeters: 2500000
Inches : 984251.968503937
Feet : 82020.99737532808

Python Programming - 2301CS404

Lab - 3

01) WAP to check whether the given number is positive or negative.

```
In [2]: num = int(input("Enter Number : "))

if (num > 0) : print(f"{num} Is Positive.")
elif (num < 0) : print(f"{num} Is Negative")
else : print(f"{num} Is Zero")
```

```
Enter Number : -4
-4 Is Negative
```

02) WAP to check whether the given number is odd or even.

```
In [4]: num = int(input("Enter Number : "))

if (num % 2 == 0) : print(f"{num} Is Even.")
else : print(f"{num} Is Odd")
```

```
Enter Number : 4
4 Is Even.
```

03) WAP to find out largest number from given two numbers using simple if and ternary operator.

```
In [16]: num = int(input("Enter Number : "))
num2 = int(input("Enter Number2 : "))

# if (num < num2) : print(f"{num2} Is Grater Than {num} .")
# else : print(f"{num} Is Grater Than {num2} .")

#or

max = (num,num2) [num < num2]
print("max : " ,max)

# or

# max = num if (num > num2) else num2
# print("max : " ,max)
```

```
Enter Number : 4
Enter Number2 : 5
```

04) WAP to find out largest number from given three numbers.

```
In [ ]: num1 = int(input("Enter Number1 : "))
num2 = int(input("Enter Number2 : "))
num3 = int(input("Enter Number3 : "))

# max = ((num1,num3) [num1 < num3],(num2,num3) [num2 < num3]) [num1 < num2]

# or

max = ( (num3) if (num3 > num2) else (num2) ) if (num1 < num3) else ( (num1) if

# or

# max_val = max(num1 , num2 , num3)

print("Max : ",max)
```

```
Enter Number1 : 73
Enter Number2 : 48
Enter Number3 : 3
Max : 73
```

05) WAP to check whether the given year is leap year or not.

[If a year can be divisible by 4 but not divisible by 100 then it is leap year but if it is divisible by 400 then it is leap year]

```
In [ ]: year = int(input("Enter Year"))

if(year % 400 == 0 or (year % 4 == 0) and not (year % 100 == 0)):
    print(year,"Is Leap Year")
else:
    print(year,"Is Not Leap Year")
```

```
Enter Year 2025
2025 Is Not Leap Year
```

06) WAP in python to display the name of the day according to the number given by the user.

```
In [ ]: day = int(input("Enter Day (1-7)"))

match (day % 7):
    case 1 : print("Monday")
    case 2 : print("Tuesday")
    case 3 : print("Wednesday")
    case 4 : print("Thursday")
    case 5 : print("Friday")
    case 6 : print("Saturday")
    case 7 : print("Sunday")
```

```
Enter Day (1-7) 9
```

Tuesday

07) WAP to implement simple calculator which performs (add,sub,mul,div) of two no. based on user input.

```
In [ ]: num = int(input("Enter Number : "))
op = input("Enter Oparater ( + , - , * , / ) : ")
num2 = int(input("Enter Number2 : "))

match op:
    case "+" : print("Sum : ", num + num2)
    case "-" : print("Tuesday", num - num2)
    case "*" : print("Wednesday", num * num2)
    case "/" : print("Thursday", num / num2)
    case _   : print("Enter Valid Input")
```

```
Enter Number : 83
Enter Oparater ( + , - , * , / ) : *
Enter Number2 : 2
Wednesday 166
```

08) WAP to read marks of five subjects. Calculate percentage and print class accordingly.

Fail below 35

Pass Class between 35 to 45

Second Class

between 45 to 60

First Class between 60 to 70

Distinction if more than 70

```
In [ ]: mark1 , mark2 , mark3 , mark4 , mark5 = input("Enter 5 Subject Marks : ").split()

mark1 = int(mark1)
mark2 = int(mark2)
mark3 = int(mark3)
mark4 = int(mark4)
mark5 = int(mark5)

percentage = (mark1 + mark2 + mark3 + mark4 + mark5) / 5

if (percentage > 70):
    print("Distinction Class")

elif (percentage > 60):
    print("First Class")

elif (percentage > 45):
    print("Second Class")

elif (percentage > 35):
```

```
print("Pass Class")

else :
    print("Fail")
```

Enter 5 Subject Marks : 44 66 88 99 33
First Class

09) WAP to find the second largest number among three user input numbers.

```
In [ ]: num1 = int(input("Enter Number1 : "))
num2 = int(input("Enter Number2 : "))
num3 = int(input("Enter Number3 : "))

if (num1 >= num2 and num1 <= num3) or (num1 <= num2 and num1 >= num3):
    second = num1

elif (num2 >= num1 and num2 <= num3) or (num2 <= num1 and num2 >= num3):
    second = num2

else:
    second = num3

print("Second Largest Number Is:", second)
```

Enter Number1 : 727
Enter Number2 : 3
Enter Number3 : 4
Second Largest Number Is: 4

10) WAP to calculate electricity bill based on following criteria. Which takes the unit from the user.

- a. First 1 to 50 units – Rs. 2.60/unit
- b. Next 50 to 100 units – Rs. 3.25/unit
- c. Next 100 to 200 units – Rs. 5.26/unit
- d. above 200 units – Rs. 8.45/unit

```
In [ ]: units = int(input("Enter the number of units consumed: "))

bill = 0.0

if units <= 50:
    bill = units * 2.60

elif units <= 100:
    bill = (50 * 2.60) + (units - 50) * 3.25

elif units <= 200:
    bill = (50 * 2.60) + (50 * 3.25) + (units - 100) * 5.26

else:
    bill = (50 * 2.60) + (50 * 3.25) + (100 * 5.26) + (units - 200) * 8.45
```

```
print(f"Total Electricity Bill = Rs. {bill:.2f}")
```

Enter the number of units consumed: 1050

Total Electricity Bill = Rs. 8001.00



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 4

Roll No.: 459

Name: Vibhu Shihora

01) WAP to print 1 to 10.

```
In [2]: for i in range(1,11):  
        print(i)
```

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

02) WAP to print 1 to n.

```
In [3]: n = int(input("Enter Number n : "))  
  
        for i in range(1,n+1):  
            print(i)
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

03) WAP to print odd numbers between 1 to n.

```
In [4]: n = int(input("Enter Number n : "))  
  
for i in range(3,n,2):  
    print(i)
```

1
3
5
7
9
11
13

04) WAP to print numbers between two given numbers which is divisible by 2 but not divisible by 3.

```
In [6]: num1 = int(input("Enter Starting Number : "))  
num2 = int(input("Enter Ending Number : "))  
  
for i in range(num1+1 , num2):  
    if(i % 2 == 0 and i % 3 != 0):  
        print(i)
```

2
4
8
10
14
16

05) WAP to print sum of 1 to n numbers.

```
In [10]: n = int(input("Enter Number : "))  
  
sum = 0  
  
for i in range(1,n+1):
```

```
    sum += i

print(sum)
```

28

06) WAP to print sum of series $1 + 4 + 9 + 16 + 25 + 36 + \dots n$.

```
In [12]: n = int(input("Enter Number : "))

sum = 0

for i in range(1,n+1):
    sum += i*i

print(sum)
```

140

07) WAP to print sum of series $1 - 2 + 3 - 4 + 5 - 6 + 7 \dots n$.

```
In [14]: n = int(input("Enter Number : "))

sum = 0

# for i in range(1,n+1):
#     if(i % 2 == 0):
#         sum -= i
#     else:
#         sum += i

sign = 1

for i in range(1,n+1):
    sum += i * sign

    sign *= -1

print(sum)
```

2

08) WAP to print Multiplication Table of the given number.

```
In [17]: n = int(input("Enter Number : "))

for i in range(1,11):
    print(n , "*" , i , "=" , n * i)
```

```

19 * 1 = 19
19 * 2 = 38
19 * 3 = 57
19 * 4 = 76
19 * 5 = 95
19 * 6 = 114
19 * 7 = 133
19 * 8 = 152
19 * 9 = 171
19 * 10 = 190

```

09) WAP to find Factorial of the given number.

```

In [18]: n = int(input("Enter Number : "))

fac = 1

for i in range(n,0,-1):
    fac *= i

print(fac)

```

120

10) WAP to print GCD of given two numbers.

```

In [6]: a = int(input("Enter Number 1 : "))
b = int(input("Enter Number 2 : "))

# while(b != 0):
#     a,b = b,a%b
# print(a)

min_n = min(a,b)

for i in range(min_n,0,-1):
    if(b % i == 0):
        print(i)
        break

```

15

11) WAP to find Factors of the given number.

```

In [9]: n = int(input("Enter Number : "))

for i in range(1,n+1):
    if(n % i == 0):
        print(i,end = " ")

```

1 2 3 6

12) WAP to find whether the given number is Prime or not.

```

In [14]: n = int(input("Enter Number : "))

for i in range(2,int(pow(n,1/2))+1): # ahiya java ni jem sqrt function nai ave s

```

```

        if(n % i == 0):
            print("No",n,"Is Not Prime Number")
            break
    else:
        print("Yes",n,"Is Prime Number")

```

No 18 Is Not Prime Number

13) WAP to print sum of digits of given number.

```

In [17]: n = int(input("Enter Number : "))

sum = 0

while(n != 0):
    sum += n%10
    n //= 10

print(sum)

```

6

14) WAP to check whether the given number is Palindrome or not.

```

In [26]: n = int(input("Enter Number : "))
original_num = n
# n2 = n[::-1]

# if(n == n2):
#     print(n,"Is Palindrome")
# else:
#     print(n,"Is Not Palindrome")
#     print(n,"Is Not Palindrome")

#or

rev = 0

while n != 0:
    rev = rev * 10 + (n % 10)
    # rev*=10+ (n % 10)=> rev=10( rev+(n % 10))
    n //= 10

if(rev == original_num):
    print(original_num,"Is Palindrome")
else:
    print(original_num,"Is Not Palindrome")

```

353 Is Palindrome

15) WAP to check whether the given number is an Armstrong Number or not.

```

In [28]: n = input("Enter Number : ")

power = len(n)

```

```

n = int(n)

original_num = n

sum = 0

while(n != 0):
    sum += (n%10) ** power
    n //= 10

if(sum == original_num):
    print(original_num,"Is Armstrong")
else:
    print(original_num,"Is Not Armstrong")

```

153 Is Armstrong

16) WAP to print all the perfect numbers between 1 to n.

In [34]:

```

n = int(input("Enter Number : "))

for i in range(2,n):
    sum = 0
    for j in range(1,i):
        if(i % j == 0):
            sum += j
    else:
        if(sum == i):
            print("Perfect Num : ",i)

```

Perfect Num : 6

Perfect Num : 28



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 5

Roll No.: 459

Name: Vibhu Shihora

1) Character Category Counter

Problem: Count uppercase, lowercase, digits, and special characters in a string.

Sample Inputs & Outputs:

- Input: Python3@Lab → Uppercase: 2, Lowercase: 7, Digits: 1, Special: 1
- Input: HELL0123 → Uppercase: 5, Lowercase: 0, Digits: 3, Special: 0
- Input: hi@#9 → Uppercase: 0, Lowercase: 2, Digits: 1, Special: 2

```
In [13]: str = input("Enter String : ")

lowercase , uppercase , digits , special = 0 , 0 , 0 , 0

for i in str:
    if(i.islower()):
        lowercase += 1
    elif(i.isupper()):
        uppercase += 1
    elif(i.isdigit()):
        digits += 1
    elif(not i.isalnum()):
        special += 1

print(f"Uppercase: {uppercase}, Lowercase: {lowercase}, Digits: {digits}, Special: {special}")
```

Enter String : hi@3#

Uppercase: 0, Lowercase: 2, Digits: 1, Special: 2

2) Reverse Each Word

- Input: Python is Easy → nohtyP si ysaE
- Input: Hello World → olleH dlrow
- Input: Learn Python Fast → nraeL nohtyP tsaF

```
In [20]: str = input("Enter String : ")

str = str[::-1]

li = str.split(" ")

li = li[::-1]

print(" ".join(li))
```

Enter String : Python is Easy
nohtyP si ysaE

3) Case Pattern Identifier

- PYTHON → Uppercase
- python → Lowercase
- Python Programming → Title Case

```
In [23]: str = input("Enter String : ")

if(str.islower()):
    print("LowerCase")
elif(str.isupper()):
    print("UpperCase")
elif(str.istitle()):
    print("TitleCase")
```

Enter String : py
LowerCase

4) First and Last Occurrence Finder

- programming , g → First:3 Last:10
- banana , a → First:1 Last:5
- hello , z → Not found

```
In [31]: str = input("Enter String : ")
ch = input("Enter ch : ")

if(str.find(ch) == -1):
    print("Not Found")
else:
    print(f"First : {str.find(ch)}") # index char no made to erro jyare find ch

    print(f>Last : {str.rfind(ch)}")

    # or here we also can use index and rindex
```

Enter String : programming
Enter ch : g

First : 3
Last : 10

5) Word Frequency Counter (ignore the case)

- Python is easy and Python is powerful → python: 2
- Java is popular but java is verbose → java: 2
- C is fast → c:1

```
In [35]: str = input("Enter String : ").lower()
word = input("Enter Word : ")

print(str.count(word))
```

Enter String : Python is easy and Python is powerful
Enter Word : python
2

6) Remove Extra Spaces

- Python is fun → Python is fun
- Hello World → Hello World
- NoExtraSpace → NoExtraSpace

```
In [52]: str = input("Enter String : ").strip()
li = str.split(" ")
li2 = []

for i in li:
    if(not (i == "")):
        li2.append(i)

print(" ".join(li2))
# print(li2)
```

Enter String : Hello World
Hello World

7) Prefix and Suffix Removal

- unhappy.txt → happy
- pretest.py → test
- unwanted.doc → wanted

```
In [ ]: s = input("Enter String : ")

prefixes = ["pre", "un"]
suffixes = [".txt", ".py", ".doc"]

# remove prefix
for p in prefixes:
    if s.startswith(p):
        s = s.removeprefix(p)
        break
```

```
# remove suffix
for suf in suffixes:
    if s.endswith(suf):
        s = s.removesuffix(suf)
        break

print(s)
```

Enter String : pretty.txt
tty

8) Replace Vowels

- Programming → Pr*gr*mm*ng
- Education → *d*c*t**n
- Sky → Sky

```
In [61]: str = input("Enter String : ")

for i in str:
    if(i.lower() in "aeiou"):
        str = str.replace(i,"*")

print(str)
```

Enter String : Programming
Pr*gr*mm*ng

9) String Compression

- aaabbccccc → a3b2c4
- xxxyyyz → x3y3z1
- abcd → a1b1c1d1

```
In [3]: st = input("Enter String : ")
s = ""
# for i in st:
#     if(s.find(i) == -1):
#         s += i + str(st.count(i))

for ch in st:

    if(ch not in s):
        s += ch + str(st.count(ch))

print(s)
```

a3b3c2

10) Toggle Case (without using str.swapcase())

- PyThOn → pYtHoN
- HELlo → heLLo
- Python → pYTHON

```
In [ ]: str = input("Enter String : ")

# print(str.swapcase())

s = ""

for i in str:
    if(i.isupper()):
        s += i.lower()
    elif(i.islower()):
        s += i.upper()
    else:
        s += i

print(s)
```

Enter String : PyThOn
pYtHoN

11) Username Validation

valid username rules:

1. only alphabets and digits
2. does not start with a digit
3. length ≥ 6

- User123 → Valid
- 123User → Invalid
- Us@12 → Invalid

```
In [ ]: username = input("Enter UserName : ")

if(username.isalnum() and not username[0].isdigit() and len(username) >= 6):
    print(f"{username} Is Valid")
else:
    print(f"{username} Is Invalid")
```

Enter UserName : Us@12
Us@12 Is Invalid

12) Palindrome Checker (ignore the case)

- Madam → Palindrome
- Level → Palindrome
- Python → Not Palindrome

```
In [ ]: word = input("Enter Word : ")

if(word.lower() == word[::-1].lower()):
    print(f"{word} Is Palindrome")
else:
    print(f"{word} Is Not Palindrome")
```

Enter Word : Python
Python Is Not Palindrome

13) Longest Word Finder

- Python programming is interesting → programming
- I love coding → coding
- Data structures and algorithms → structures

```
In [ ]: s = input("Enter Sentence : ")

li = s.split(" ")

ans = ""

for i in li:
    if(len(ans) < len(i)):
        ans = i

print("Longest Word Is",ans)
```

Enter Sentence : Data structures and algorithms
Longest Word Is structures

14) Case Conversion

- python programming
 - Upper: PYTHON PROGRAMMING
 - Lower: python programming
 - Title: Python Programming
 - Capitalize: Python programming

```
In [ ]: s = input("Enter Sentence : ")

print("Upper : ",s.upper())
print("Lower : ",s.lower())
print("Title : ",s.title())
print("Capitalize : ",s.capitalize())
```

Enter Sentence : python programming
Upper : PYTHON PROGRAMMING
Lower : python programming
Title : Python Programming
Capitalize : Python programming

15) Custom Split

- apple,banana,grape
- 10|20|30
- A-B-C-D

```
In [ ]: data = input("Enter Data : ")
sp = input("Split By : ")

print(data.split(sp))
```

```
Enter Data : A-B-C-D  
Split By : -  
['A', 'B', 'C', 'D']
```

Python Programming - 2301CS404

Lab - 6

Roll No. : 459

Name : Vibhu Shihora

01) WAP to find sum of all the elements in a List.

```
In [4]: list = [1,3,5,7]

sum = 0

for i in list:
    sum += i

print("Sum : " , sum)
```

Sum : 16

02) WAP to find largest element in a List.

```
In [6]: list = [2,4,5]

# print(max(list))

# or

mx = list[0]
for i in list:
    if mx < i:
        mx = i

print(mx)
```

5

03) WAP to find the length of a List.

```
In [8]: list = [1,3,5,6]

count = 0

for i in list:
    count+=1

print("Len : ",count)
```

Len : 4

04) WAP to interchange first and last elements in a list.

```
In [10]: list = [1,3,5,6]

temp = list[0]
list[0] = list[-1]
list[-1] = temp

print(list)
```

[6, 3, 5, 1]

05) WAP to split the List into two parts and append the first part to the end.

```
In [16]: list = [1,3,4,5,6]

p = int(input("Enter Position To Split : "))

left = list[:p+1]
right = list[p+1:]

list = right + left

print(list)
```

Enter Position To Split : 2
[5, 6, 1, 3, 4]

06) WAP to interchange the elements on two positions entered by a user.

```
In [11]: list = [1,3,5,6]

p1 = int(input("Enter Position 1 : "))
p2 = int(input("Enter Position 2 : "))

temp = list[p1]
list[p1] = list[p2]
list[p2] = temp

print(list)
```

Enter Position 1 : 1
Enter Position 2 : 3
[1, 6, 5, 3]

07) WAP to reverse the list entered by user.

```
In [ ]: n = int(input("Enter Length : "))

list = []

for i in range(n):
    list.append(int(input(f"Enter {i+1} Element :")))

list = list[::-1]

print(list)
```

```
Enter Length : 4
Enter 1 Element : 2
Enter 2 Element : 3
Enter 3 Element : 5
Enter 4 Element : 5
[2, 3, 5, 5]
```

08) WAP to print even numbers in a list.

```
In [23]: list = [1,2,3,4]

for i in list:
    if(i % 2 == 0):
        print(i)
```

```
2
4
```

09) WAP to count unique items in a list.

```
In [31]: list = [11,11,4,4,66]

unique = 0

for i in list:
    if(list.count(i) == 1):
        unique += 1

print(unique)
```

```
1
```

10) WAP to copy a list.

```
In [35]: list = [1,2,3,4,5]

list2 = list.copy()

# or

# list2 = list # Refrance Same

print(list2)
```

Out[35]: [1, 2, 3, 4, 5]

11) WAP to print all odd numbers in a given range.

```
In [39]: a = int(input("Enter a : "))
b = int(input("Enter b : "))

for i in range(a,b+1):
    if(i % 2 != 0):
        print(i)
```

Enter a : 5

Enter b : 10

5

7

9

12) WAP to count occurrences of an element in a list.

```
In [46]: list = [11,2,2,3,4,5,55,6,11,11,55,11]

for i in set(list):
    print(f"number {i} : occurrence {list.count(i)}")
```

number 2 : occurrence 2

number 3 : occurrence 1

number 4 : occurrence 1

number 5 : occurrence 1

number 6 : occurrence 1

number 11 : occurrence 4

number 55 : occurrence 2

13) WAP to find second largest number in a list.

```
In [50]: list = [1,2,4,6,7]

# for i in list:
#     if(max(list) != i):
#         list2.append(i)

# print(max(list2))

list.sort() # Must Be Remember Its Not Return List

print(list[-2])
```

6

14) WAP to extract elements with frequency greater than K.

```
In [52]: list = [11,2,2,3,4,5,55,6,11,11,55,11]

k = int(input("Enter K : "))

for i in set(list):
```

```
if(list.count(i) > k):  
    print(f"number {i} : occurrence {list.count(i)}")
```

Enter K : 1
number 2 : occurrence 2
number 11 : occurrence 4
number 55 : occurrence 2

15) WAP to create a list of squared numbers from 0 to 9 with and without using List Comprehension.

```
In [56]: list = []  
  
for i in range(0,10):  
    list.append(i*i)  
  
# or comprehencen  
  
list = [i*i for i in range(0,10)]  
  
print(list)
```

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

16) WAP to create a new list (fruit whose name starts with 'b') from the list of fruits given by user.

```
In [58]: n = int(input("Enter Length : "))  
  
list = []  
  
for i in range(n):  
    list.append(input(f"Enter {i+1} Element :"))  
  
fruit = []  
  
for i in list:  
    if(i[0] == "b"):  
        fruit.append(i)  
  
print(fruit)
```

Enter Length : 3
Enter 1 Element : apple
Enter 2 Element : banana
Enter 3 Element : mengo
['banana']

17) WAP to create a list of common elements from given two lists.

```
In [59]: list = [1,2,3,4]  
list2 = [1,3,6,8,7]  
list3 = []  
  
for i in list:  
    if(list2.count(i) != 0):  
        list3.append(i)
```

```
print(list3)
```

```
[1, 3]
```



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 7

Roll No : 459

Name : Vibhu Shihora

01) WAP to find sum of tuple elements.

Sample Input:

T = (10, 20, 30, 40)

Sample Output:

Sum of tuple elements = 100

```
In [1]: T = (10,20,30,40)
sum = 0
for i in T:
    sum+=i

print("Sum of tuple elements =",sum)
```

Sum of tuple elements = 100

02) WAP to find Maximum and Minimum K elements in a given tuple.

Sample Input:

T = (7, 2, 9, 4, 1, 5)
K = 2

Sample Output:

Minimum 2 elements: (1, 2)
Maximum 2 elements: (9, 7)

```
In [12]: T = (7, 2, 9, 4, 1, 5)
k = 2

T = tuple(sorted(T,reverse=True)) #descending and aa list ma return kare

minTup = (T[-1:-k-1:-1])
maxTup = (T[:k])

print("Minimum 2 elements:",minTup)
print("Maximum 2 elements:",maxTup)
```

Minimum 2 elements: (1, 2)
Maximum 2 elements: (9, 7)

03) WAP to find tuples which have all elements divisible by K from a list of tuples.

Sample Input:

```
tuple_list = [(2, 4, 6), (3, 6, 9), (4, 8, 12), (5, 10, 15)]
K = 2
```

Sample Output:

Tuples divisible by 2 : [(2, 4, 6), (4, 8, 12)]

```
In [17]: tuple_list = [(2, 4, 6), (3, 6, 9), (4, 8, 12), (5, 10, 15)]
k = 2

li = []

# for i in range(len(tuple_list)): #we also use without range
#     flag = False
#     for j in tuple_list[i]:
#         if(j % k != 0):
#             flag = True
#             break
#     if(not flag):
#         li.append(tuple_list[i])

# or

for i in tuple_list:

    for j in i:
        if(j % k != 0):
            break
    else:
        li.append(i)

print(li)
```

[(2, 4, 6), (4, 8, 12)]

04) WAP to create a list of tuples from given list having number and its cube in each tuple.

Sample Input:

```
numbers = [1, 2, 3, 4, 5]
```

Sample Output:

List of tuples with number and its cube: [(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]

```
In [3]: numbers = [1, 2, 3, 4, 5]

t = tuple( (i , i ** 3) for i in numbers )

print(t)
```

((1, 1), (2, 8), (3, 27), (4, 64), (5, 125))

05) WAP to find tuples with all positive elements from the given list of tuples.

Sample Input:

```
tuple_list = [(1, 2, 3), (-1, 2, 3), (4, 5, 6), (0, 1, 2), (-3, -4, -5)]
```

Sample Output:

Tuples with all positive elements: [(1, 2, 3), (4, 5, 6)]

```
In [20]: tuple_list = [(1, 2, 3), (-1, 2, 3), (4, 5, 6), (0, 1, 2), (-3, -4, -5)]

li = []

for i in tuple_list:

    for j in i:
        if(j <= 0):
            break
    else:
        li.append(i)

print("Tuples with all positive elements:",li)
```

[(1, 2, 3), (4, 5, 6)]

06) WAP to remove tuples of length K.

Sample Input:

```
tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
K = 2
```

Sample Output:

List after removing tuples of length 2 : [(3, 4, 5), (6,), (7, 8, 9)]

```
In [ ]: tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
K = 2

for i in tuple_list:
    if(len(i) == k):
        tuple_list.remove(i)

print("List after removing tuples of length 2 :",tuple_list)
```

List after removing tuples of length 2 : [(3, 4, 5), (6,), (7, 8, 9)]

07) WAP to remove all occurrences of a given element from a tuple.

Sample Input:

```
T = (3, 5, 3, 7, 3, 9)
x = 3
```

Sample Output:

(5, 7, 9)

```
In [36]: T = (3, 5, 3, 7, 3, 9)
x = 3

# T = list(T)

# T.remove(x) # Aa ekj element remove karse

T = tuple(i for i in T if i != x)

print(T)
```

(5, 7, 9)

08) WAP to remove duplicates from tuple.

Sample Input:

```
T = (3, 5, 3, 7, 3, 9)
```

Sample Output:

(3, 5, 7, 9)

```
In [25]: T = (3, 5, 3, 7, 3, 9)

T = tuple(set(T))

print(T)
```

(9, 3, 5, 7)

09) WAP to multiply adjacent elements of a tuple and print that resultant tuple.

Sample Input:

t = (2, 3, 4, 5)

Sample Output:

Resultant tuple after multiplying adjacent elements: (6, 12, 20)

```
In [37]: t = (2, 3, 4, 5)
tp = []
for i in range(len(t)-1):
    tp.append(t[i]*t[i+1])

print(tuple(tp))
```

(6, 12, 20)

10) WAP to test if the given tuple is distinct or not.

Sample Input:

tuple1 = (1, 2, 3, 4)
tuple2 = (1, 2, 2, 3)

Sample Output:

(1, 2, 3, 4) is distinct? -> True
(1, 2, 2, 3) is distinct? -> False

```
In [5]: # tuple1 = (1, 2, 3, 4)
tuple1 = (1, 2, 2, 3)

if(len(set(tuple1)) != len(tuple1)):
    print(False)

else:
    print(True)
```

False

11) WAP to rotate the elements of a tuple to the right by k positions.

Sample Input:

```
T = (10, 20, 30, 40, 50)
k = 2
```

Sample Output:

```
(40, 50, 10, 20, 30)
```

```
In [43]: T = (10, 20, 30, 40, 50)
k = 2

tp = []

for i in range(k+1, len(T)):
    tp.append(T[i])

for i in range(k+1):
    tp.append(T[i])

print(tuple(tp))
```

```
(40, 50, 10, 20, 30)
```

12) WAP to merge two tuples of equal length alternately.**Sample Input:**

```
T1 = (1, 3, 5)
T2 = (2, 4, 6)
```

Sample Output:

```
(1, 2, 3, 4, 5, 6)
```

```
In [45]: T1 = (1, 3, 5)
T2 = (2, 4, 6)

tp = []

if(len(T1) == len(T2)):
    for i in range(len(T1)):
        tp.append(T1[i])
        tp.append(T2[i])

print(tuple(tp))
```

```
(1, 2, 3, 4, 5, 6)
```

13) WAP to create a tuple of squares of elements that are even and greater than 10.**Sample Input:**

T = (4, 12, 7, 18, 10)

Sample Output:

(144, 324)

```
In [48]: T = (4, 12, 7, 18, 10)

tp = tuple(i*i for i in T if (i%2==0 and i > 10))

print(tp)

(144, 324)
```

14) WAP to create (index, value) pairs for prime numbers.

Sample Input:

T = (4, 7, 9, 11, 15)

Sample Output:

((1, 7), (3, 11))

```
In [61]: T = (4, 7, 9, 11, 15)

def isPrime(n):
    for i in range(2, int(pow(n, 0.5)+1)):
        if n % i == 0:
            return False
    else:
        return True

tp = []

for i in range(len(T)):
    if(isPrime(T[i])):
        tp.append(tuple([i, T[i]]))

print(tp)
```

[(1, 7), (3, 11)]

15) WAP to split a tuple into even-index and odd-index tuples.

Sample Input:

T = ('a', 'b', 'c', 'd', 'e')

Sample Input:

even = ('a', 'c', 'e')
odd = ('b', 'd')

```
In [54]: T = ('a', 'b', 'c', 'd', 'e')
```

```
even = []
odd = []

for i in range(len(T)):
    if i % 2 == 0:
        even.append(T[i])
    else:
        odd.append(T[i])
print("even =", even)
print("odd =", odd)
```

```
even = ['a', 'c', 'e']
odd = ['b', 'd']
```

16) WAP to compute column-wise sum from a tuple of lists.

Sample Input:

```
T = ([1, 2, 3], [4, 5, 6], [7, 8, 9])
```

Sample Output:

```
12 15 18
```

```
In [59]: T = ([1, 2, 3], [4, 5, 6], [7, 8, 9])
sm = 0
for i in range(len(T)):
    for j in range(len(T[0])):
        sm += T[j][i]
    print(sm)
```

```
12
27
45
```

Python Programming - 2301CS404

Lab - 8

Roll No : 459

Name : Vibhu Shihora

01) WAP to find their union, intersection, and difference of the given two sets.

Sample Input:

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}

```
In [1]: a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a.union(b)) # or a | b
print(a.intersection(b)) # or a & b
print(a.difference(b)) # or a - b
```

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
```

02) WAP to Find elements common to all three given sets.

Sample Input:

A = {1, 2, 3, 4}
B = {2, 3, 5}
C = {2, 3, 6}

Sample Output:

{2, 3}

```
In [5]: a = {1, 2, 3, 4}
b = {2, 3, 5}
c = {2, 3, 6}

# print((a & b) & c)

# or

print(a.intersection(b.intersection(c)))
```

{2, 3}
{2, 3}

03) WAP to remove duplicate elements from a list while preserving unique values.

Sample Input:

[10, 20, 10, 30, 20, 40]

Sample Output:

{10, 20, 30, 40}

```
In [9]: li = [10, 20, 10, 30, 20, 40]

s = set(li)

print(s)
```

{40, 10, 20, 30}

Out[9]: {10, 20, 30, 40}

04) WAP to count how many elements are common between two sets.

Sample Input:

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

```
In [10]: A = {1, 2, 3, 4}
         B = {3, 4, 5, 6}

         print(len(a&b))
```

2

05) WAP to Merge two dictionaries. If a key exists in both dictionaries, add their values.

Sample Input:

```
Dict1 = {1: 10, 2: 20, 3: 30}
Dict2 = {2: 40, 3: 50, 4: 60}
```

Sample Output:

```
{1: 10, 2: 60, 3: 80, 4: 60}
```

```
In [ ]: Dict1 = {1: 10, 2: 20, 3: 30}
         Dict2 = {2: 40, 3: 50, 4: 60}

         for k,v in Dict1.items():
             if k in Dict2:
                 Dict2[k] += v
             else:
                 Dict2[k] = v

         print(Dict2)

         # result = Dict2.copy()

         # for k, v in Dict1.items():
         #     result[k] = result.get(k, 0) + v

         # print(result)
```

```
{2: 60, 3: 80, 4: 60, 1: 10}
```

06) WAP to Sort a dictionary in ascending order of its values.

Hint: in sorted(), use udf for key parameter

Sample Input:

```
{'Math': 70, 'Physics': 85, 'Chemistry': 60}
```

Sample Output:

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

```
In [49]: Dict = {'Math': 70, 'Physics': 85, 'Chemistry': 90}

def insort(item):
    return item[1]

di = sorted(Dict.items(), key=insort)

# or

# di2 = sorted(Dict.items(), key=lambda x : x[1])

# print(dict(di2))

print(dict(di))
```

```
{'Chemistry': 90, 'Physics': 85, 'Math': 70}
```

07) WAP to find the keys having maximum and minimum values in a dictionary.

Sample Input:

```
{'A': 50, 'B': 90, 'C': 30}
```

Sample Output:

```
Max Value Key: B
Min Value Key: C
```

```
In [68]: Dict = {'A': 50, 'B': 90, 'C': 30}

di = sorted(Dict.items(), key=lambda x : x[1])
di = dict(di)

print("max",list(di.keys())[len(di)-1])
print("min",list(di.keys())[0])

# or

# m = max(Dict , key=Dict.get)
# n = min(Dict , key=Dict.get)
```

```
max B
min C
```

08) WAP to create a dictionary with numbers from 1 to N as keys and their squares as values.

Sample Input:

```
N = 5
```

Sample Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
In [71]: Dict = {i : i*i for i in range(1,6,1)}  
print(Dict)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

09) WAP to count the frequency of each character in a string using a dictionary.

Sample Input:

```
programming
```

Sample Output:

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

```
In [ ]: s = "programming"  
  
# di = {}  
  
# for i in s:  
#     di[i] = s.count(i)  
  
di = {i : s.count(i) for i in s}  
  
print(di)
```

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

10) WAP to swap keys and values of a dictionary.

Sample Input:

```
{1: 'A', 2: 'B', 3: 'C'}
```

Sample Output:

```
{'A': 1, 'B': 2, 'C': 3}
```

```
In [76]: di = {1: 'A', 2: 'B', 3: 'C'}  
  
di2 = {j : i for i , j in di.items()}  
  
print(di2)
```

```
{'A': 1, 'B': 2, 'C': 3}
```

11) WAP to find the student with the highest total marks, given student names and their marks.

Sample Input:

```
{'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}
```

Sample Output:

Topper: Amit

```
In [ ]: stu = {'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}

# for k , v in stu.items():
#     stu[k] = sum(v)

# stu = sorted(stu.items(), key=lambda x : x[1],reverse=True)

stu = dict(sorted(stu.items(),key=lambda x : sum(x[1])))

print("Topper :",stu[0][0])
```

Topper : Amit

12) WAP to extract keys whose values are even numbers.

Sample Input:

```
{1: 11, 2: 20, 3: 15, 4: 40}
```

Sample Output:

```
[2, 4]
```

```
In [85]: di = {1: 11, 2: 20, 3: 15, 4: 40}

li = [k for k,v in di.items() if v % 2 == 0]

print(li)
```

Out[85]: [2, 4]

13) WAP to Find common keys between two dictionaries.

Sample Input:

```
Dict1 = {1: 'A', 2: 'B', 3: 'C'}
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}
```

Sample Output:

```
[2, 3]
```

```
In [86]: Dict1 = {1: 'A', 2: 'B', 3: 'C'}
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}
```

```
li = []

for i in Dict1.keys():
    if i in Dict2:
        li.append(i)

print(li)
```

Out[86]: [2, 3]

14) WAP to handle missing keys in dictionaries.

Sample Input - 1:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}
key = 'c'
```

Sample Output - 1:

8

Sample Input - 2:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}
key = 'd'
```

Sample Output - 2:

Key Not Found

```
In [ ]: dict1 = {'a': 5, 'c': 8, 'e': 2}
key = 'c'

# if key in dict1:
#     print(dict1[key])
# else:
#     print("Not Found")

dict1.get(key, "Key Not Found")
```

8

15) WAP to create a dictionary using dictionary comprehension from two lists containing student names and marks. Include only those students in the dictionary whose marks are greater than or equal to 75.

Sample Input:

```
students = ['Amit', 'Riya', 'Neha', 'Karan']
marks = [78, 72, 85, 60]
```

Sample Output:

```
{'Amit': 78, 'Neha': 85}
```

```
In [92]: students = ['Amit', 'Riya', 'Neha', 'Karan']  
marks = [78, 72, 85, 60]  
  
di = {k:v for k,v in zip(students,marks) if v >= 75}  
  
print(di)
```

```
Out[92]: {'Amit': 78, 'Neha': 85}
```

Python Programming - 2301CS404

Lab - 9

Roll No : 459

Name : Vibhu Shihora

**01) Write a function to calculate BMI given mass and height.
(BMI = mass/h**2)**

```
In [7]: mass , h = input("Enter Mass And h : ").split(" ")
mass = float(mass)
h = float(h)

def calBMI(mass , h):
    BMI = mass / h ** 2

    return BMI

res = calBMI(mass,h)

print(res)
```

Enter Mass And h : 40 5
1.6

02) Write a function that add first n numbers.

```
In [8]: n = int(input("Enter Number n : "))

def printFirstN(n):
    add = 0
    for i in range(1,n+1):
        add += i

    return add

print(printFirstN(n))
```

Enter Number n : 5
15

03) Write a function that returns 1 if the given number is Prime or 0 otherwise.

```
In [16]: n = int(input("Enter Number : "))

def isPrime(n):
    for i in range(2,int(n**0.5)+1):
        if(n % i == 0):
            return 0

    return 1

print(isPrime(n))
```

Enter Number : 7
1

04) Write a function that returns the list of Prime numbers between given two numbers.

```
In [17]: a , b = input("Enter Range a And b : ").split(" ")
a = int(a)
b = int(b)

def isPrime(n):
    for i in range(2,int(n**0.5)+1):
        if(n % i == 0):
            return 0

    return 1

def primeRange(a,b):
    li = []

    for i in range(a,b+1):
        if(isPrime(i) == 1):
            li.append(i)

    return li

print(primeRange(a,b))
```

Enter Range a And b : 3 19
[3, 5, 7, 11, 13, 17, 19]

05) Write a function that returns True if the given string is Palindrome or False otherwise.

```
In [19]: s = input("Enter String : ")

checkPalindrome = lambda x : x == x[::-1]

print(checkPalindrome(s))
```

Enter String : oko

True

06) Write a function that returns the sum of all the elements of the list.

```
In [ ]: li = input("Enter list : ").split(" ")

li = list(map(int,li))

sumList = lambda x : sum(x)

print(sumList(li))
```

Enter list : 1 2 3 4
10

07) Write a function to calculate the sum of the first element of each tuples inside the list.

```
In [2]: li = [(1,2),(3,4),(5,6)]

def sumListTuple(li):
    s = 0
    for i in li:
        s += i[0]
    return s

print(sumListTuple(li))
```

9

08) Write a recursive function to find nth term of Fibonacci Series.

```
In [5]: n = int(input("Enter nTh Term : "))

# 0 1 1 2 3 5 8 13

def fibonacciNth(n):
    if(n == 0 or n == 1):
        return n
    return fibonacciNth(n-2) + fibonacciNth(n-1)

fibonacciNth(n-1)
```

Enter nTh Term : 5

Out[5]: 3

09) Write a function to get the name of the student based on the given rollno.

Example: Given dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'} find name of student whose rollno = 103

```
In [7]: dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'}

def getName(dict1,roll):
    return dict1.get(roll)

print(getName(dict1,103))
```

Jay

10) Write a function to get the sum of the scores ending with zero.

Example : scores = [200, 456, 300, 100, 234, 678]

Ans = 200 + 300 + 100 = 600

```
In [9]: scores = [200, 456, 300, 100, 234, 678]

def endWithZero(scores):
    s = 0
    for i in scores:
        if(i % 10 == 0):
            s += i
    return s

print(endWithZero(scores))
```

600

11) Write a function to invert a given Dictionary.

hint: keys to values & values to keys

Before : {'a': 10, 'b':20, 'c':30, 'd':40}

After : {10:'a', 20:'b', 30:'c', 40:'d'}

```
In [10]: di = {'a': 10, 'b':20, 'c':30, 'd':40}

def invert(di):
    return { v : k for k , v in di.items()}

print(invert(di))
```

{10: 'a', 20: 'b', 30: 'c', 40: 'd'}

12) Write a function that returns the number of uppercase and lowercase letters in the given string.

example : Input : s1 = AbcDEfgh ,Oupthput : no_upper = 3, no_lower = 5

```
In [13]: s1 = "AbcDEfgh"

def countUpperCase(s1):
    no_upper = 0
    no_lower = 0
```

```

    for ch in s1:
        if(ch.isupper()):
            no_upper += 1
        else:
            no_lower += 1

    return no_upper , no_lower

print(countUpperCase(s1))

```

(3, 5)

13) Write a lambda function to get smallest number from the given two numbers.

```

In [18]: a , b = input("Enter 2 Number : ").split(" ")

a = int(a)
b = int(b)

small = lambda a,b:(a,b)[b<a]

small(a,b)

```

Enter 2 Number : 4 5

Out[18]: 4

14) For the given list of names of students, extract the names having more that 7 characters. Use filter().

```

In [21]: names = ["hey" , "python" , "ok" , "okokokk" , "parisssss"]

res = list(filter(lambda x : len(x) >= 7,names))

res

```

Out[21]: ['okokokk', 'parisssss']

15) For the given list of names of students, convert the first letter of all the names into uppercase. use map().

```

In [22]: names = ["hey" , "python" , "ok" , "okokokk" , "parisssss"]

res = list(map(lambda x : x.title(),names))

res

```

Out[22]: ['Hey', 'Python', 'Ok', 'Okokokk', 'Parisssss']

16) Write udfs to call the functions with following types of arguments:

1. Positional Arguments
2. Keyword Arguments

3. Default Arguments
4. Variable Length Positional(*args) & variable length Keyword Arguments (**kwargs)
5. Keyword-Only & Positional Only Arguments

```
In [34]: def one(a,b):
          print(a,b)

          one(1,4)

          def two(**kw):
              print(kw)

          two(name="vi",age=19)

          def three(c,d=0):
              print(c,d)

          three(5)

          def four(*li , **kw):
              print("li : ",li)
              print("kw : ",kw)

          four(1,2,3,name="vi",age=19)

          def five(e,f,/,*,g,h):
              print(e,f,g,h)

          five(10,20,g=90,h=100)

1 4
{'name': 'vi', 'age': 19}
5 0
li :  (1, 2, 3)
kw :  {'name': 'vi', 'age': 19}
10 20 90 100
```

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 10

Roll No : 459

Name : Vibhu Shihora

01) WAP to read and display the contents of a text file. (also try to open the file in some other directory)

- in the form of a string
- line by line
- in the form of a list

```
In [10]: fp = open("f1.txt" , "r")

# print(fp.read) # Its Read Whole File

n = fp.readline()
while(n != ''):
    print(n,end="")
    n = fp.readline()

# print(fp.readlines()) #ALL The Lines As List

fp.close()
```

Hello Jay
Ok
By

02) WAP to create file named "new.txt" only if it doesn't exist.

```
In [12]: fp = open("f2.txt", "w")
```

```
fp.close()
```

03) WAP to read first 5 lines from the text file.

```
In [19]: fp = open("f1.txt", "r")

count = 0

while(count != 5):
    print(fp.readline(), end="")
    count += 1

fp.close()
```

Hello Jay
Ok
By
line 4
line 5

04) WAP to find the longest word(s) in a file.

```
In [21]: fp = open("f1.txt", "r")

words = fp.read().split(" ")

longestWord = ""

for w in words:
    if(len(longestWord) < len(w)):
        longestWord = w

print(longestWord)

fp.close()
```

DarshanUniversity

05) WAP to count the no. of lines, words and characters in a given text file.

```
In [30]: fp = open("f1.txt", "r")

n = fp.readline()

count = 0

while(n != ''):
    count += 1
    n = fp.readline()

fp.seek(0,0)

totalWords = len(fp.read().split(" "))

fp.seek(0,0)
```

```

totalChar = len(list(fp.read()))

print("Total Numbers Of Lines : " , count)
print("Total Numbers Of Words : " , totalWords)
print("Total Numbers Of Chars : " , totalChar)

fp.close()

```

Total Numbers Of Lines : 8
Total Numbers Of Words : 6
Total Numbers Of Chars : 71

06) WAP to copy the content of a file to the another file.

```

In [35]: fp = open("f1.txt", "r")

fp2 = open("f2.txt", "w+")

content = fp.read()

fp2.write(content)
fp2.seek(0)
print(fp2.read())

fp.close()

fp2.close()

```

Hello Jay DarshanUniversity
Ok
By
line4
line5
ok2 (line6)
line7
line8

07) WAP to find the size of the text file.

```

In [36]: fp = open("f1.txt", "r")

print(len(fp.read()))

```

71

08) WAP to create an UDF named frequency to count occurrences of the specific word in a given text file.

```

In [47]: def frequency(words):
        return words.count("hey")

fp = open("f2.txt", "r")

print(frequency(fp.read().split()))

# fp.seek(0)
# print(fp.read().split())

```

```
fp.close()
```

3

```
['Hello', 'Jay', 'DarshanUniversity', 'Ok', 'hey', 'By', 'line4', 'line5', 'ok2',  
'(line6', 'hey', 'line7', 'line8', 'hey']
```

09) WAP to get the score of five subjects from the user, store them in a file. Fetch those marks and find the highest score.

```
In [21]: fp = open("f2.txt", "w+")  
  
marks = input("Enter Subject Marks").split(" ")  
  
# print(marks)  
  
for i in range(0, len(marks)):   
    fp.write("Marks Of Subject"+str(i)+":"+marks[i]+"\\n")  
  
fp.seek(0)  
  
li = []  
  
for i in fp:  
    print(int(i.split(":")[-1][0:2]))  
    li.append(int(i.split(":")[-1][0:2]))  
  
print("max : ", max(li))  
  
# or  
  
# n = fp.readline().split(":")[-1]  
  
# while(n != ''):  
#     print(n, end="")  
#     n = (fp.readline().split(":"))[-1]  
  
fp.close()
```

77

88

55

99

99

10) WAP to write first 100 prime numbers to a file named primenumbers.txt

(Note: each number should be in new line)

```
In [1]: def isPrime(n):  
  
    if(n < 2):  
        return False  
  
    for i in range(2 , int(pow(n,0.5)) +1):
```

```

        if(n % i == 0):
            return False
        else:
            return True

with open("primenumbers.txt","w") as fp:
    count = 0
    n = 2

    fp.write("No\tPrimeNo\n")

    while(count != 100):
        if(isPrime(n)):
            fp.write(str(count+1) + "\t"+str(n) + "\n")
            count += 1
        n += 1

```

11) WAP to merge two files and write it in a new file.

```

In [4]: fp = open("f1.txt","r")
        fp2 = open("f2.txt","r")
        fp3 = open("f3.txt","w")

        fp3.write(fp.read() + "\n" + fp2.read())

        fp.close()
        fp2.close()
        fp3.close()

```

12) WAP to replace word1 by word2 of a text file. Write the updated data to new file.

```

In [7]: fp = open("f1.txt" , "r")
        fp2 = open("f4.txt" , "w")

        fp2.write(fp.read().replace("hello","hey"))

        fp.close()
        fp2.close()

```

13) Demonstrate tell() and seek() for all the cases(seek from beginning-end-current position) taking a suitable example of your choice.

```

In [16]: with open("demo.txt", "w") as f:
        f.write("ABCDEFGHJKLMNOPQRSTUVWXYZ")

        # binary mode mate b
        fp = open("demo.txt", "rb")

        # 1. From Beginning (seek(offset,0))
        print("Initial position:", fp.tell())

        fp.seek(5, 0)      # move 5 bytes from beginning

```

```

print("After seek(5,0):", fp.tell())

print("Reading next char:", fp.read(1))    # Should read F

# 2. From Current Position (seek(offset,1))
fp.seek(3, 1)    # move 3 bytes forward from current
print("\nAfter seek(3,1):", fp.tell())

print("Reading next char:", fp.read(1))    # Should read J

# 3. From End (seek(offset,2))
fp.seek(-4, 2)    # move 4 bytes backward from end
print("\nAfter seek(-4,2):", fp.tell())

print("Reading next char:", fp.read(1))    # Should read W

fp.close()

```

Initial position: 0
 After seek(5,0): 5
 Reading next char: b'F'

After seek(3,1): 9
 Reading next char: b'J'

After seek(-4,2): 22
 Reading next char: b'W'



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 11

Roll No : 459

Name : Vibhu Shihora

Progrmas based on Python Data Structure + UDF + File Handling

Practical 1: Count Lines, Words and Characters

Problem Definition:

Write a Python program to read text from a file and count the total number of lines, words, and characters. The result should be written to an output file.

```
In [2]: fp = open("input1.txt" , "r")

totalChar = len(fp.read())

fp.seek(0)

totalWords = len(fp.read().split())

fp.seek(0)

totalLines = len(fp.readlines())

fp2 = open("output1.txt", "w")

fp2.write("Total Character : " + str(totalChar) + "\n" + "Total Words : " + str(

fp.close()
fp2.close()
```

Practical 2: Search a Word in a File

Problem Definition:

Write a Python program to search a given word in a text file and count its occurrences. Store the result in an output file.

```
In [6]: fp = open("input1.txt" , "r")
fp2 = open("output2.txt","w")

word = input("Enter Word To Search In Input1.txt File :")

fp2.write(word + " : " + str(fp.read().split().count(word)) + " Times")

fp2.close()
fp.close()
```

Enter Word To Search In Input1.txt File : heyy

Practical 3: Student Marks and Average

Problem Definition:

Write a Python program to read roll number, name, and marks of students from a file. Store marks in a list, calculate the average using a user-defined function, and write student details along with average marks to an output file.

```
In [23]: fp = open("marks.txt" , "r")
fp2 = open("output3.txt" , "w")

rollNos = []
names = []
marks = []

def avg(li):
    return str(sum(li)/len(li))

for i in fp:
    rollNos.append(i[0:3])
    names.append(i[4:9])
    marks.append(i[10 : ].split())

print(rollNos)
print(names)
print(marks)

n = len(marks)

ans = []

for i in marks:
    ans.append(list(map(int , i)))

print(ans)

fp.seek(0)
n = len(fp.readlines())
fp.seek(0)
```

```

for i in range(0,n):
    fp2.write("Name : " + names[i] + "\nRollNo : " + str(rollNos[i]) + "\nAvg :

fp.close()
fp2.close()

```

```

['101', '102', '103']
['Vibhu', 'Jay ', 'Mohit']
[['99', '97', '98', '97'], ['99', '99', '98', '97'], ['100', '98', '96', '94']]
[[99, 97, 98, 97], [99, 99, 98, 97], [100, 98, 96, 94]]

```

Practical 4: Maximum and Minimum using Tuple

Problem Definition:

Write a Python program to read numbers from a file, store them in a tuple, and find the maximum and minimum values. Write the result to an output file.

```

In [5]: fp = open("input4.txt","r")
        fp2 = open("output4.txt","w")

        tp = ()

        data = fp.read().split()

        print(data)

        for i in data:
            tp += (int(i),)

        print(tp)

        fp2.write("Max : "+str(max(tp)) + "\nMin : "+str(min(tp)))

        fp.close()
        fp2.close()

```

```

['10', '20', '80', '90', '77']
(10, 20, 80, 90, 77)

```

Practical 5: Remove Duplicate Words using Set

Problem Definition:

Write a Python program to read words from a file and remove duplicate words using a set. Store the unique words in an output file.

```

In [13]: fp = open("input1.txt","r")
        fp2 = open("output5.txt","w")

        # st = set()

        # data = fp.read().split()

        # for i in data:
        #     st.add(i)

```

```

# print(st)

# for i in st:
#     fp2.write(i + " ")

fp.seek(0)

st = set()

for i in fp:
    words = i.split(" ")
    for w in words:
        st.add(w)

print(st)

for i in st:
    fp2.write(i + " ")

fp.close()
fp2.close()

```

```
{'hey\n', 'input', 'file.txt', 'good', 'afternoon', 'ok', 'okk\n', 'by\n', '\n',
'hello', 'its'}
```

Practical 6: Word Frequency using Dictionary

Problem Definition:

Write a Python program to count the frequency of each word in a text file using a dictionary and write the result to an output file.

```

In [17]: fp = open("input1.txt","r")
         fp2 = open("output6.txt","w")

         di = {}

         data = fp.read().split()

         fp.seek(0)

         for i in data:
             di[i] = data.count(i)

         fp2.write(str(di))

         fp.close()
         fp2.close()

```

Practical 7: Student Result and Grade Calculation

Problem Definition:

Write a Python program to read roll number, name, and marks of students from a file. Use a user-defined function to determine grade (A/B/C/Fail) and write the student result to an output file.

Grade Criteria:

- Marks ≥ 75 : Grade A
- Marks ≥ 60 : Grade B
- Marks ≥ 40 : Grade C
- Marks < 40 : Fail

```
In [20]: fp = open("marks.txt" , "r")
fp2 = open("output7.txt" , "w")

def grade(m):
    if m >= 75:
        return "A"
    elif m >= 60:
        return "B"
    elif m >= 40:
        return "C"
    else:
        return "Fail"

rollNos = []
names = []
marks = []

for i in fp:
    rollNos.append(i[0:3])
    names.append(i[4:9])
    marks.append(i[10 : ].split())

print(rollNos)
print(names)
print(marks)

n = len(marks)

ans = []

for i in marks:
    ans.append(list(map(int , i)))

print(ans)

fp.seek(0)
n = len(fp.readlines())
fp.seek(0)

for i in range(0,n):

    fp2.write("Name : " + names[i] + "\nRollNo : " + str(rollNos[i]) + "\nGrade

    for g in ans[i] :
        fp2.write(grade(g) + " ")

    fp2.write("\n\n\n")

fp.close()
fp2.close()
```

```
['101', '102', '103']  
['Vibhu', 'Jay ', 'Mohit']  
[['99', '97', '98', '97'], ['99', '99', '98', '97'], ['100', '98', '96', '94']]  
[[99, 97, 98, 97], [99, 99, 98, 97], [100, 98, 96, 94]]
```



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 12

Roll No : 459

Name : Vibhu Shihora

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [2]: #ZeroDivision Error

try:
    ans = 10 / 0

except ZeroDivisionError as msg:
    print(msg)

# Value Error

try:
    b = int("abc")

except ValueError as msg:
    print(msg)

# Type Error
```

```
try:
    print(10 + "90")

except TypeError as msg:
    print(msg)
```

division by zero

invalid literal for int() with base 10: 'abc'

unsupported operand type(s) for +: 'int' and 'str'

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

```
In [7]: #Index Error

try :
    li = [1,2,3]
    print(li[10])
except IndexError as msg:
    print(msg)

#Key Error

try:
    di = {1 : "vi" , 2 : "jay"}
    print(di[10])

except KeyError as msg:
    print("Error : " , msg)
```

list index out of range

Error : 10

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```
In [8]: #FileNotFound Error

try:
    fp = open("ok.txt","r")

except FileNotFoundError as msg :
    print(msg)

#ModuleNotFound Error

try:
    import module

except ModuleNotFoundError as msg :
    print(msg)
```

```
[Errno 2] No such file or directory: 'ok.txt'  
No module named 'module'
```

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```
In [9]: # Handle ALL Exception  
  
try:  
    import module  
  
except Exception as msg :  
    print(msg)
```

No module named 'module'

05) WAP to demonstrate else and finally block.

```
In [11]: try:  
    print(10 + 0)  
except Exception as msg:  
    print(msg)  
else:  
    print("No Exception Found")  
finally:  
    print("Program Completed")
```

```
10  
No Exception Found  
Program Completed
```

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```
In [18]: def divide(a,b):  
  
    try:  
        ans = a / b  
    except ZeroDivisionError as msg :  
        print(msg)  
    else:  
        return ans  
  
print(divide(10,0))
```

```
division by zero  
None
```

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

```
In [30]: class NegativeValue(Exception):  
    pass  
  
try:  
    li = []
```

```

total = 0

for i in range(1,5):
    a = int(input("Enter Element : "))

    if(a < 0):
        raise NegativeValue("Negative Mark")

    li.append(a)
    total += a
except Exception as msg:
    print(msg)
else:
    print(total)
finally:
    print(li)

```

Negative Mark

[99]

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```

In [33]: try:
        s = input("Enter Grade sep by (,) : ").split(",")
        s = [int(i) for i in s]

        except Exception as msg:
            print(msg)

```

invalid literal for int() with base 10: 'h'

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```

In [38]: try:
        li = []
        total = 0

```

```

for i in range(1,6):
    a = int(input("Enter Markd : "))

    assert 0 <= a <= 100 , "Marks should be between 0 and 100."

    li.append(a)
    total += a

except Exception as msg:
    print(msg)

finally:
    print("li : ",li)
    print("Total : " , total / len(li))

```

Marks should be between 0 and 100.

li : [88, 90]

89.0

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```

In [50]: class WeakPasswordError(Exception):
        pass

    try:
        pas = input("Enter PassWord : ")

        countDig = 0

        for i in pas:
            if i.isdigit():
                countDig += 1
        print(countDig)

        if(len(pas) < 8 or countDig != 1):
            raise WeakPasswordError("Invalid")

    except Exception as err:
        print(err)
    else:
        print("Ok")

```

1

Ok

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```
In [54]: class NegativeValue(Exception):  
        pass  
  
        try:  
            a = int(input("Enter Element : "))  
  
            if(a < 0):  
                raise NegativeValue("Negative")  
  
            li.append(a)  
            total += a  
        except Exception as msg:  
            print(msg)  
        else:  
            print(pow(a,0.5))  
        finally:  
            print("Complete Program")
```

Negative

Complete Program



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 13

Roll No.: 459

Name: Vibhu Shihora

01) WAP to create Calculator module which defines functions like add, sub, mul and div.

Create another .py file that uses the functions available in Calculator module.

```
In [2]: import calc

print("add : " , calc.add(10 , 20) )
print("sub : " , calc.sub(10 , 20) )
print("div : " , calc.div(10 , 20) )
print("mul : " , calc.mul(10 , 20) )
```

```
add : 30
sub : -10
div : 0.5
mul : 200
```

02) WAP to pick a random character from a given String.

```
In [5]: import random

s = "helloGoodMorning"

print( random.choice( list(s) ))

# or

print( s[ random.randint(0 , len(s)-1 ) ] )
```

o
o

03) WAP to pick a random element from a given list.

```
In [15]: import random

li = [1,5,7,9,0]

print( random.choice( li ))
```

7

04) WAP to roll a dice in such a way that every time you get the same number.

```
In [21]: import random

dice = list(range(1,7) )

random.seed(5)

print( random.choice(dice) )
```

6

05) WAP to generate 3 random integers between 100 and 999 which is divisible by 5.

```
In [12]: import random

count = 0

while count != 3:

    n = random.randint(100,999)

    if(n % 5 == 0):
        print(n)
        count += 1
```

480

525

395

06) WAP to generate 100 random lottery tickets and pick two lucky tickets from it and announce them as Winner and Runner up respectively.

```
In [3]: import random

tickets = []

for i in range(100):

    n = random.randint(1000,9999)

    tickets.append(n)
```

```
winner , runnerUp = random.sample(tickets,2)

# print(tickets)
print("Winner : " , winner)
print("runnerUp : " , runnerUp)
```

Winner : 8238
runnerUp : 7737

07) WAP to print current date and time in Python.

```
In [ ]: import datetime

print("Date And Time : ", datetime.datetime.now())

# seprate

print("Date : " ,datetime.datetime.now().date())
print("Time : " ,datetime.datetime.now().time())
```

Date And Time : 2026-03-04 11:49:49.675629
Date : 2026-03-04
Time : 11:49:49.676032

08) Subtract a week (7 days) from a given date in Python.

```
In [17]: from datetime import datetime , timedelta

date = datetime(2026, 3, 10)

# print(date)

newDate = date - timedelta(days=7)

print("Original Date:", date.date())
print("Date after subtracting 7 days:", newDate.date())
```

Original Date: 2026-03-10
Date after subtracting 7 days: 2026-03-03

09) WAP to Calculate number of days between two given dates.

```
In [24]: from datetime import datetime , timedelta

d1 = datetime(2026,3,10)
d2 = datetime(2026,3,20)

diff = d2 - d1

print(diff.days)
```

10

10) WAP to Find the day of the week of a given date.(i.e. wether it is sunday/monday/tuesday/etc.)

```
In [11]: import datetime

dt = datetime.datetime.now().date()    # Its Means Current Date
day = dt.strftime("%A")

print("Day of the week:", day)
```

Day of the week: Wednesday



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 14

Roll No.: 459

Name: Vibhu Shihora

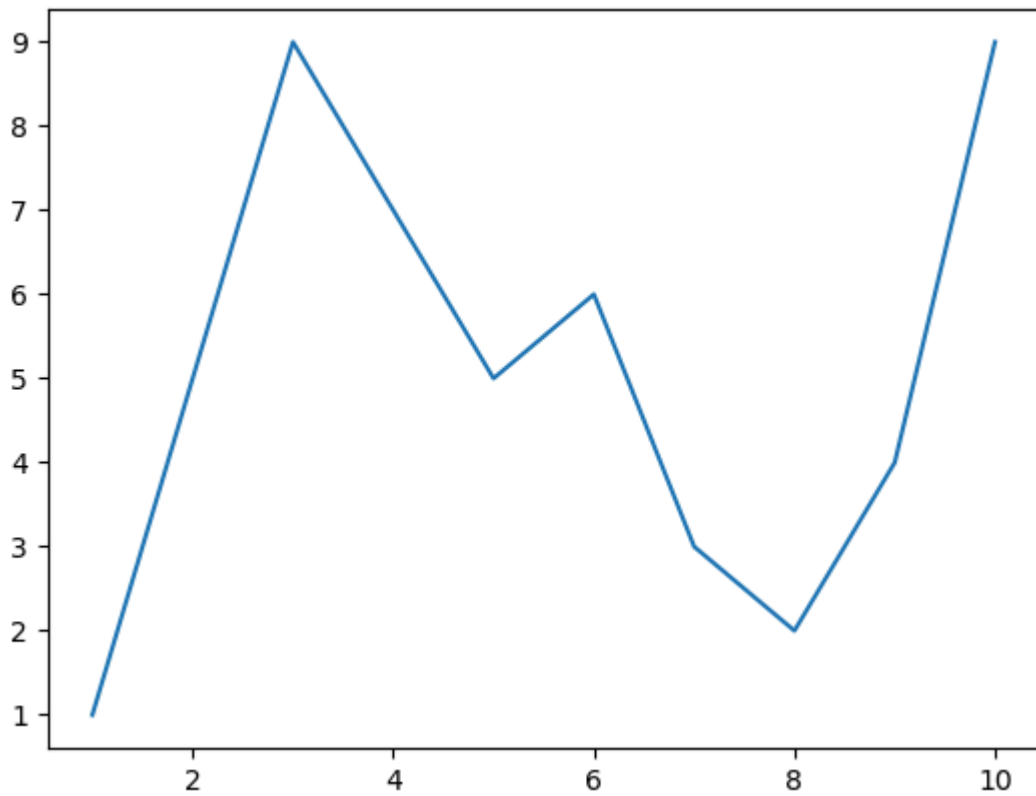
Import matplotlib below

```
In [2]: import matplotlib.pyplot as plt
```

01) WAP to display the line chart for given data for x and y axis.

```
In [ ]: x = range(1,11)  
y = [1,5,9,7,5,6,3,2,4,9]
```

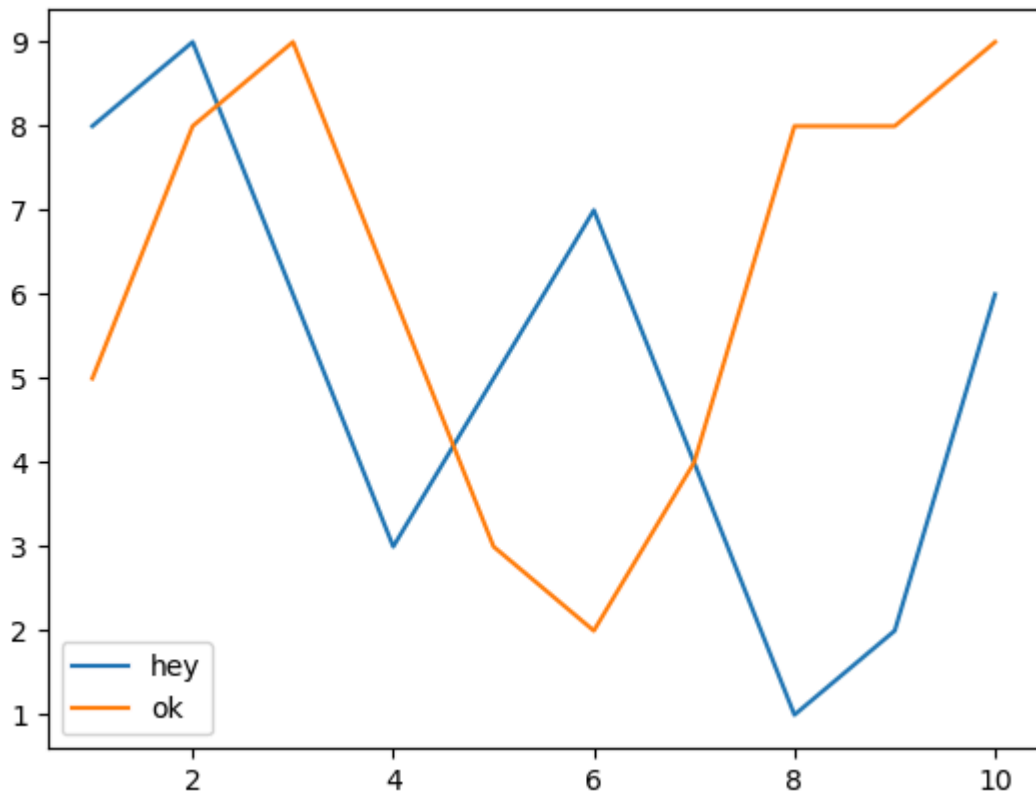
```
In [ ]: plt.plot(x,y)  
plt.show()
```



02) WAP to generate line chart for given data.

```
In [ ]: x = range(1,11,1)
cxMarks= [8,9,6,3,5,7,4,1,2,6]
cyMarks= [5,8,9,6,3,2,4,8,8,9]
```

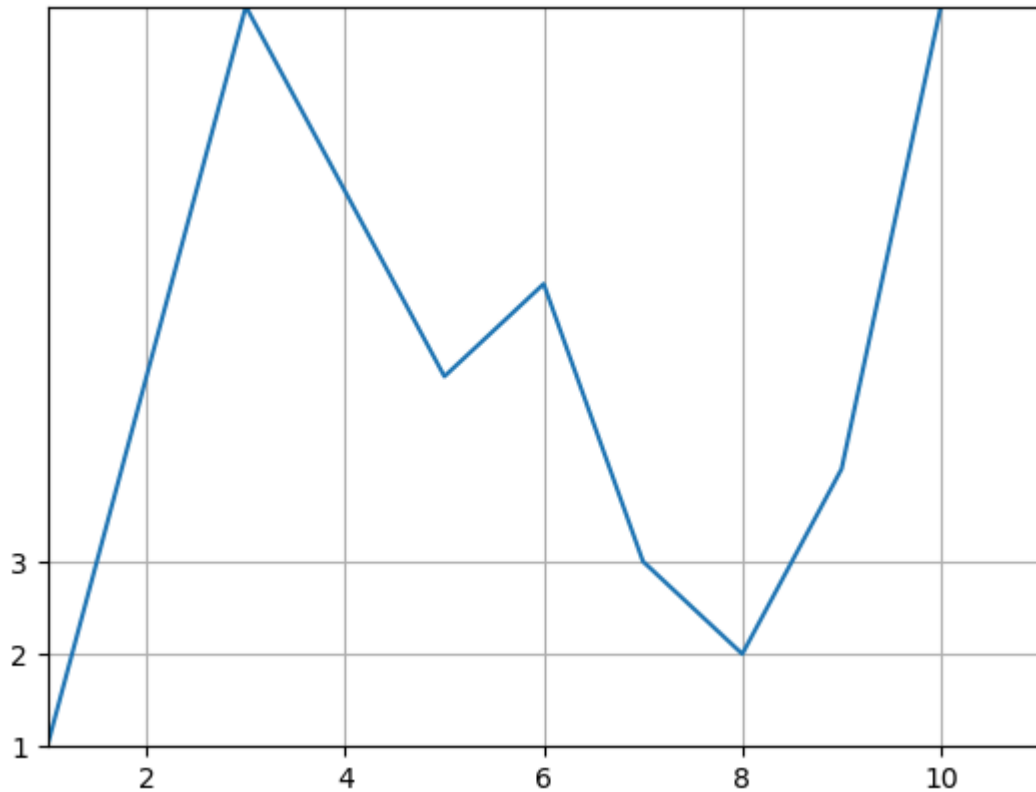
```
In [ ]: plt.plot(x,cxMarks,label="hey")
plt.plot(x,cyMarks,label="ok")
plt.legend()
plt.show()
```



03) WAP to generate line chart for given data, but create your custom axis.

```
In [ ]: x = range(1,11)
        y = [1,5,9,7,5,6,3,2,4,9]
```

```
In [ ]: ax = plt.axes()
        ax.set_xlim(1,11)
        ax.set_ylim(1,9)
        # ax.set_xticks(range(1,5))
        ax.set_yticks(range(1,4))
        ax.grid()
        plt.plot(x,y)
        plt.show()
```



04) WAP to plot a line chart showing temperature variation during a week. Mark the lowest and highest temperature.

```
In [ ]: Days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
Temperature = [20, 32, 31, 33, 34, 43, 36]
```

```
In [ ]: maxVal = max(Temperature)
maxDay = Days[Temperature.index(maxVal)]

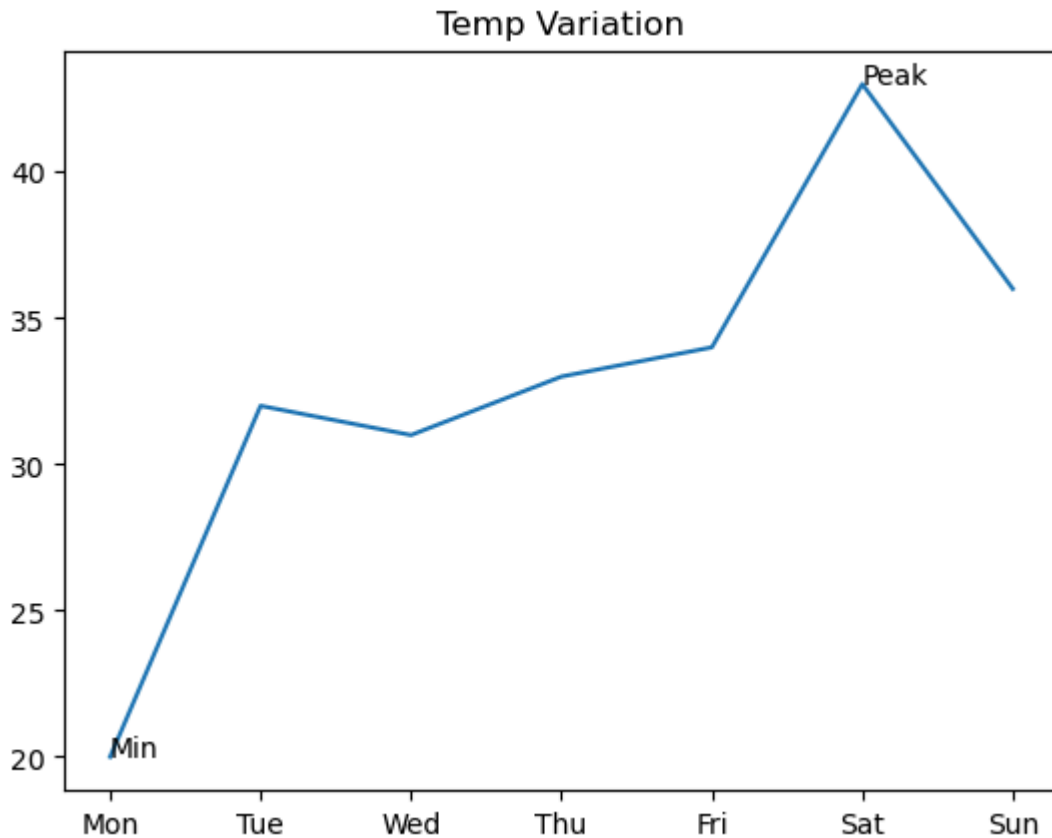
minVal = min(Temperature)
minDay = Days[Temperature.index(minVal)]

plt.plot(Days, Temperature)
plt.title("Temp Variation")

plt.annotate("Peak", xy=[maxDay, maxVal])
plt.annotate("Min", xy=[minDay, minVal])

# or

# plt.annotate('Shifted', xy=("Sat", 43), xytext=(3, 40),
#               arrowprops=dict(arrowstyle="->"))
plt.show()
```

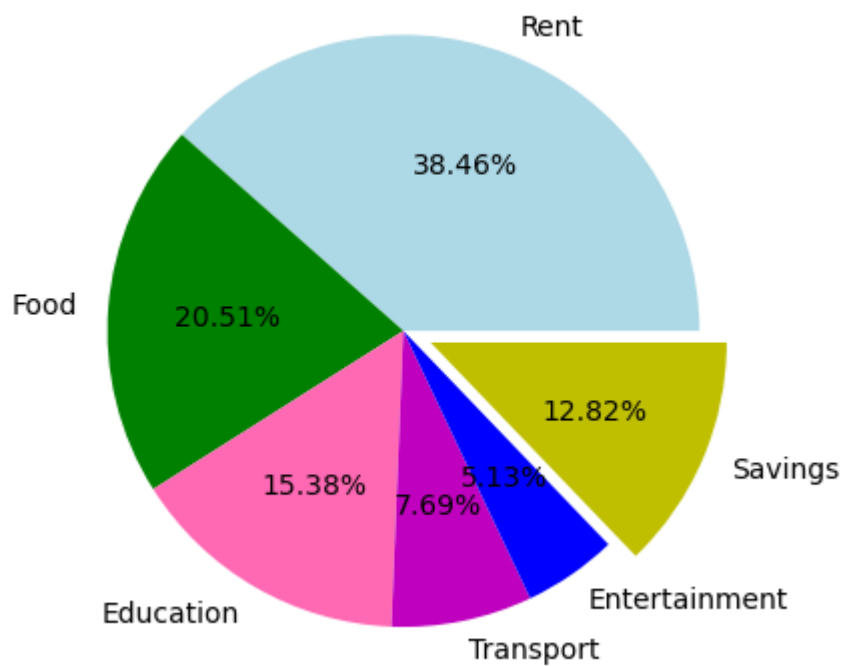


05) WAP to create a detailed pie chart representing the monthly expenditure distribution of a family.

The chart should display labels, percentage values, title, different colors for each slice, and explode one slice to highlight the highest expense.

```
In [ ]: Categories = ["Rent", "Food", "Education", "Transport", "Entertainment", "Saving"]  
Amount = [15000, 8000, 6000, 3000, 2000, 5000]
```

```
In [ ]: plt.pie(Amount, labels=Categories, autopct = "%1.2f%%", colors = ["lightblue",  
plt.show()
```

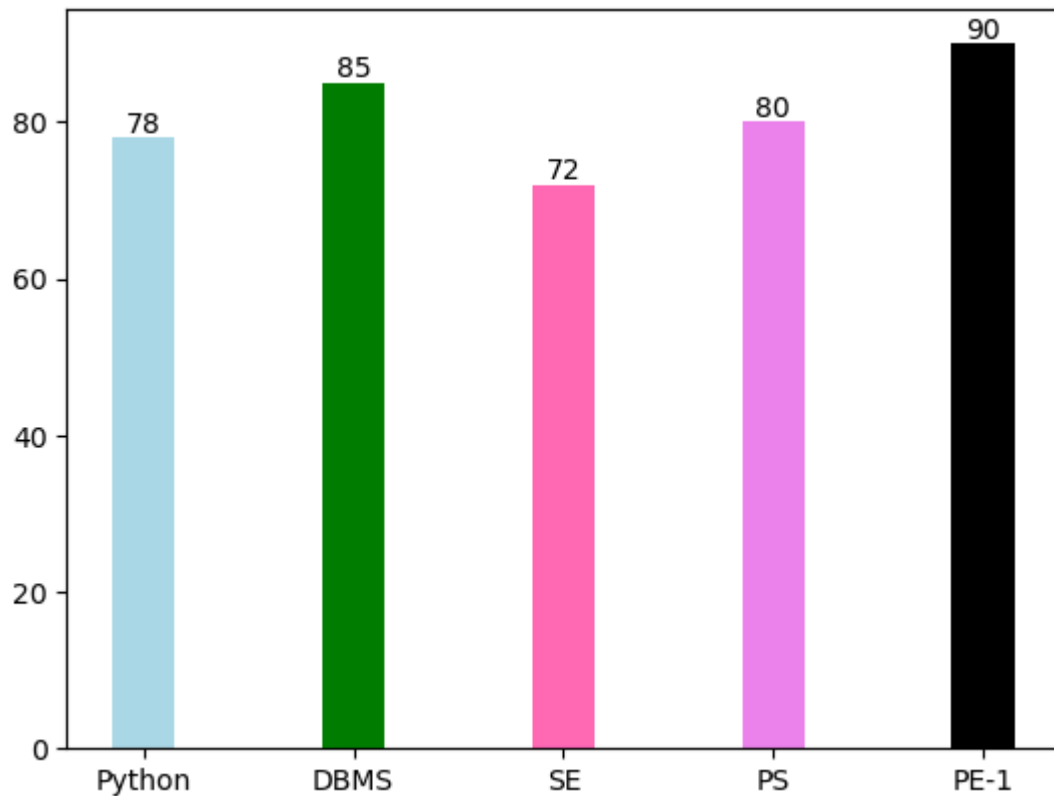


06) WAP to create bar chart showing the marks obtained by students in different subjects.

The program should also display the marks on the top of the respective bar.

```
In [ ]: Subjects = ["Python", "DBMS", "SE", "PS", "PE-1"]  
Marks = [78, 85, 72, 80, 90]
```

```
In [ ]: bars = plt.bar(Subjects, Marks, width=0.3, color = ["lightblue", "g", "hotpink", "viol  
plt.bar_label(bars)  
plt.show()
```

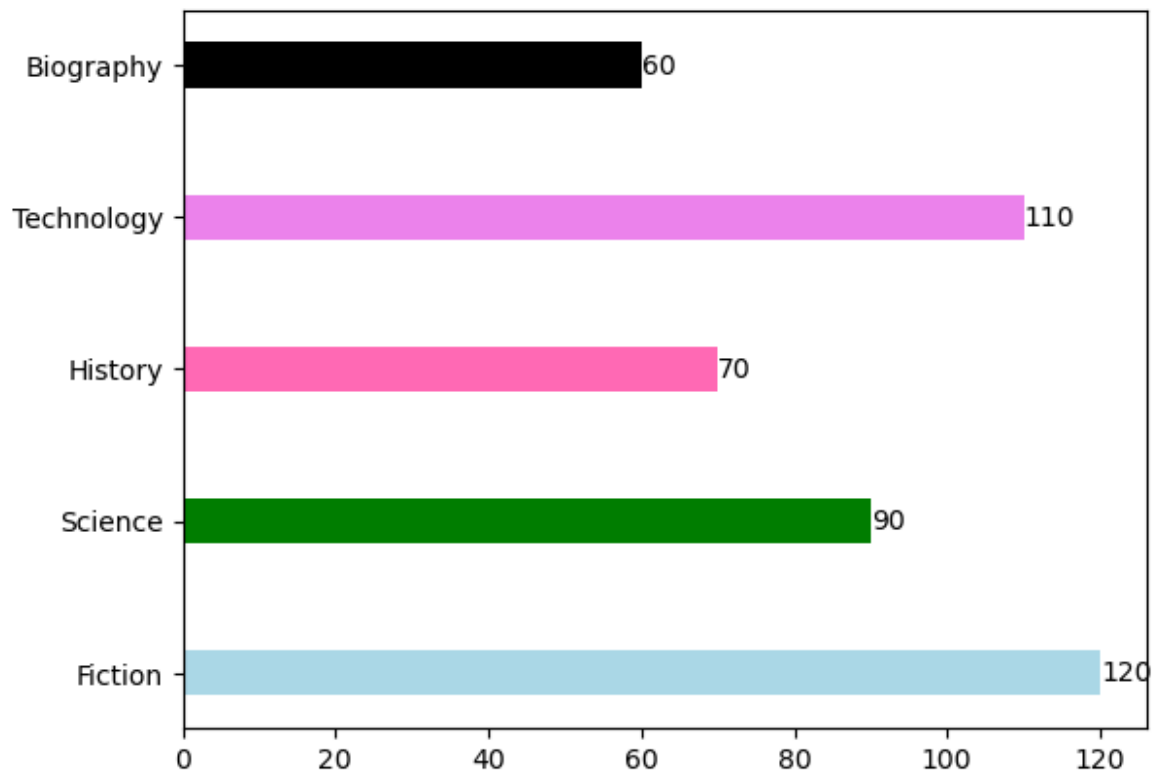


07) WAP to create a horizontal bar chart representing the number of books sold in different categories.

The program should also display the number of books sold at the end of the respective bar.

```
In [3]: Categories = ["Fiction", "Science", "History", "Technology", "Biography"]  
BooksSold = [120, 90, 70, 110, 60]
```

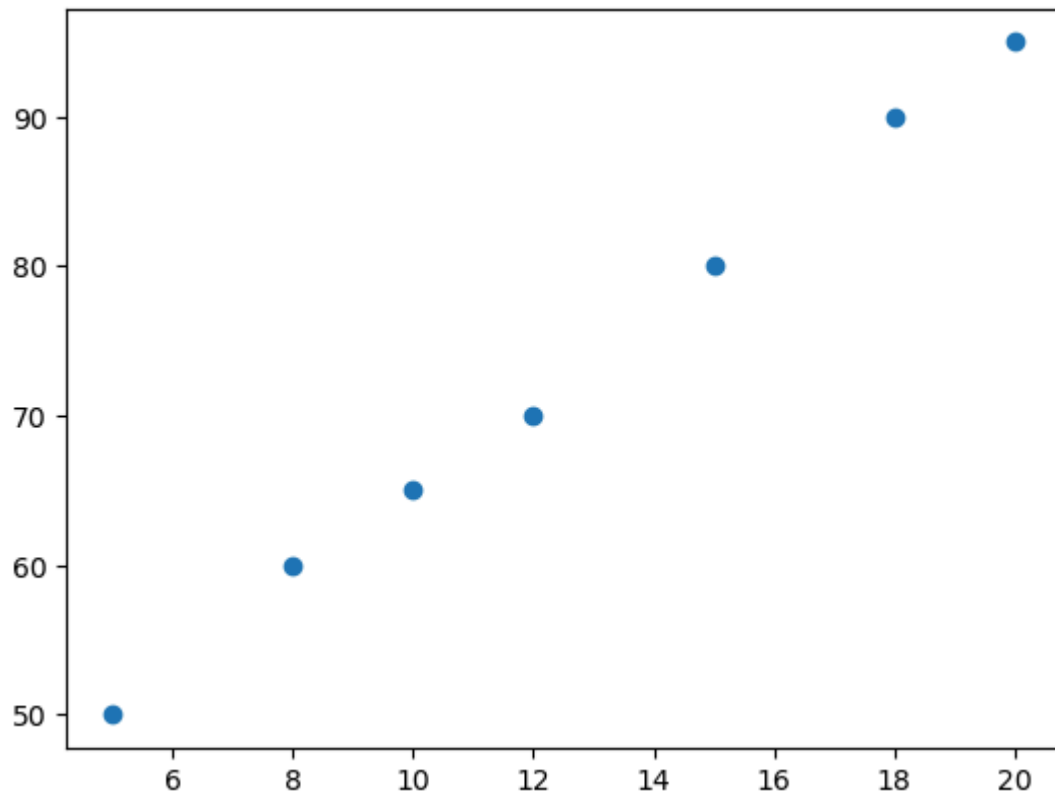
```
In [8]: bars = plt.barh(Categories, BooksSold, height=0.3, color = ["lightblue", "g", "hotpin  
plt.bar_label(bars)  
plt.show()
```



08) WAP to create a scatter plot representing the relationship between advertising expenditure and sales revenue of a company.

```
In [4]: Advertising = [5, 8, 10, 12, 15, 18, 20]
SalesRevenue = [50, 60, 65, 70, 80, 90, 95]
```

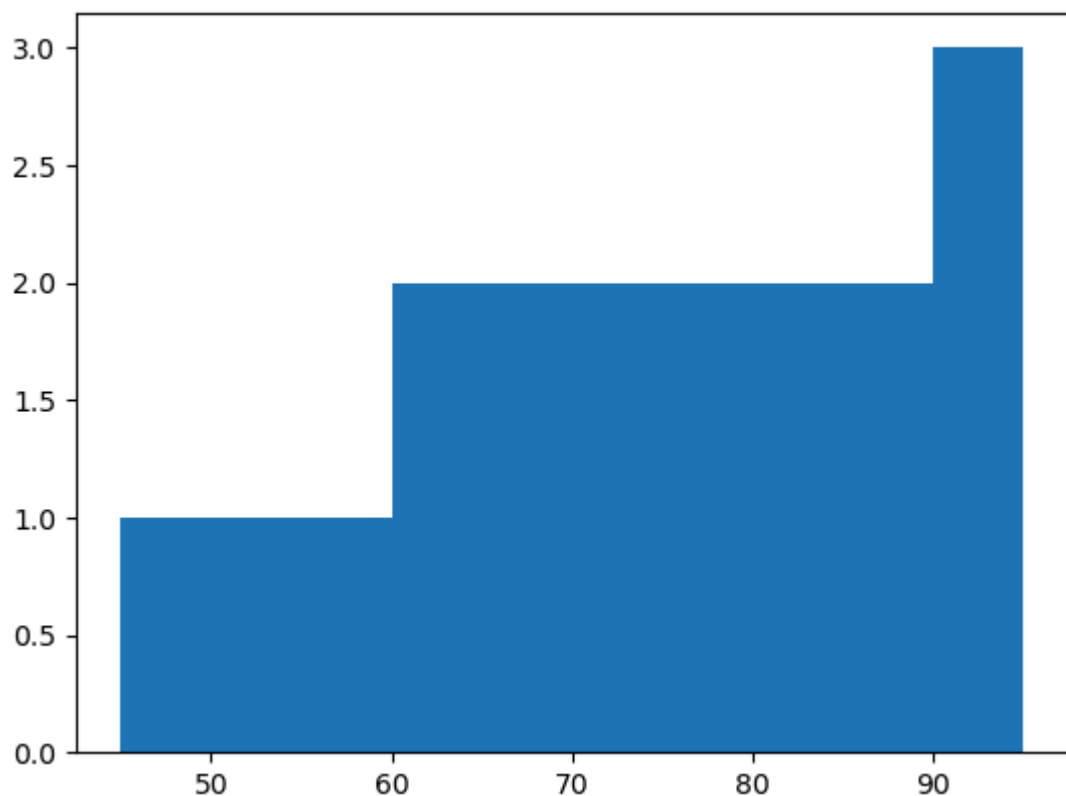
```
In [5]: plt.scatter(Advertising, SalesRevenue)
plt.show()
```



09) WAP to create a histogram representing the distribution of marks obtained by students in a class.

```
In [6]: Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]
```

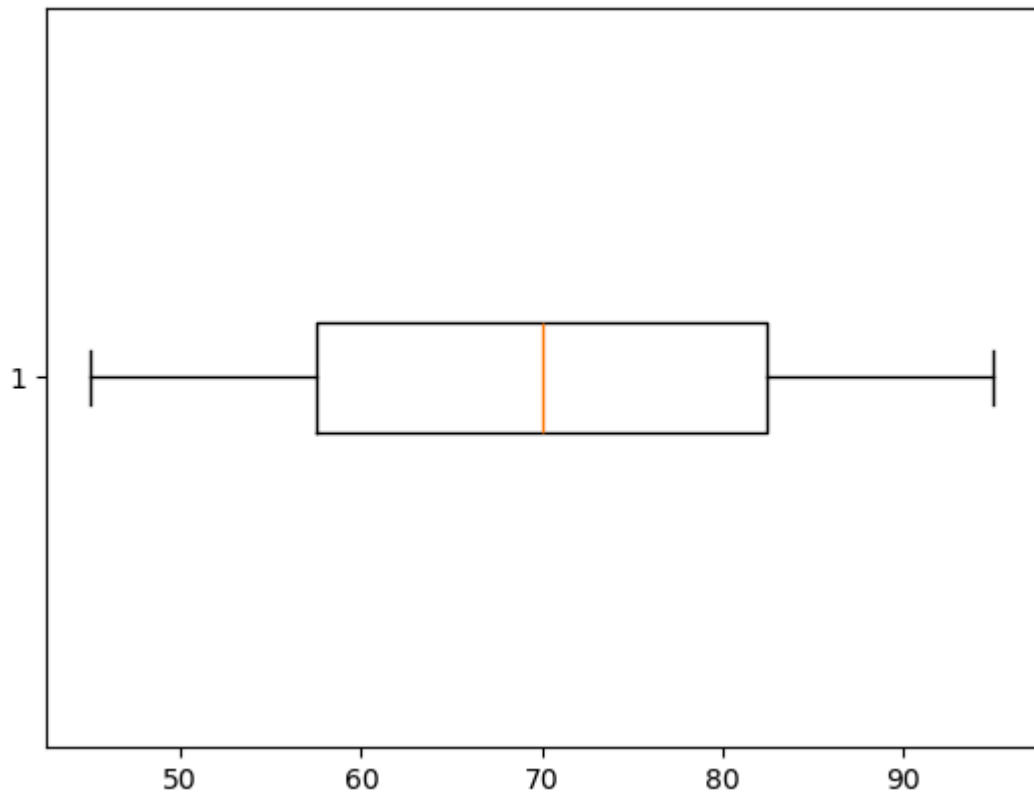
```
In [7]: plt.hist(Marks)  
plt.show()
```



10) WAP to create a box plot representing the distribution of daily commuting time (in minutes) of employees and identify outliers (if any).

```
In [15]: CommutingTime = [45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
```

```
In [18]: plt.boxplot(CommutingTime, vert=False)  
plt.show()
```



```
In [ ]:
```



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 459

Name: Vibhu Shihora

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

```
In [1]: class Students :
        def __init__(self,name,age,grade) :
            self.name = name
            self.age = age
            self.grade = grade

        def __str__(self):
            return f" name : {self.name}\n age : {self.age}\n grade : {self.grade}"

s1 = Students("vi",19,"A++")
print(s1)
```

```
name : vi
age : 19
grade : A++
```

02) Create a class named Bank_Account with Account_No, User_Name, Email,Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

```
In [7]: class Bank_Account :

        # def __init__(self,accNo,name,email,accType,accBal) :
        #     self.accNo = accNo
        #     self.name = name
        #     self.email = email
```

```

#     self.accType = accType
#     self.accBal = accBal

def getAccDetails(self):
    arr = input("Enter accNo,name,email,accType,accBal : ").split(" ")
    self.accNo = arr[0]
    self.name = arr[1]
    self.email = arr[2]
    self.accType = arr[3]
    self.accBal = arr[4]

def displayAccDetails(self):
    print("accNo : " , self.accNo)
    print("Name : " , self.name)
    print("email : " , self.email)
    print("accType : " , self.accType)
    print("accBal : " , self.accBal)

# b1 = Bank_Account("7","vi","vi@gmail.com","saving","9999999999")
def main():
    b1 = Bank_Account()
    b1.getAccDetails()
    b1.displayAccDetails()

main()

```

```

accNo : 7
Name : vi
email : sh
accType : sav
accBal : 999

```

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

```

In [13]: import math

class Circle :
    def __init__(self,r) :
        self.r = r

    def area(self):
        return math.pi * self.r * self.r
    def perimeter(self):
        return 2 * math.pi * self.r

c1 = Circle(7)
c1.area()
c1.perimeter()

```

```

Out[13]: 43.982297150257104

```

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and

display employee information.

```
In [16]: class Employees :

    def __init__(self,name,age,salary,accBal) :
        self.name = name
        self.age = age
        self.salary = salary
        self.accBal = accBal

    def displayEmp(self):
        print("Name :" , self.name)
        print("Age :" , self.age)
        print("Salary :" , self.salary)
        print("AccBal :" , self.accBal)

    def update(self,name,age,salary,accBal) :
        self.name = name
        self.age = age
        self.salary = salary
        self.accBal = accBal

e1 = Employees("vi",19,99999,7777777)

e1.displayEmp()
e1.update("vibhu",19,99999,9999999)
print("\n")
e1.displayEmp()
```

```
Name : vi
Age : 19
Salary : 99999
AccBal : 7777777
```

```
Name : vibhu
Age : 19
Salary : 99999
AccBal : 9999999
```

05) Create a bank account class with methods to deposit, withdraw, and check balance.

```
In [23]: class Bank:

    def __init__(self):
        self.bal = 5000

    def deposit(self,amount):
        self.bal += amount
        print("Deposit : ",amount)

    def withdraw(self,amount):
        self.bal -= amount
        print("Withdraw : ",amount)

    def displayBal(self):
        print("Current : ",self.bal)
```

```
b1 = Bank()
b1.deposit(100)
b1.withdraw(100)
b1.displayBal()
```

Deposit : 100
Withdraw : 100
Current : 5000

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

```
In [25]: class inventory:
        def __init__(self,name,price,quantity):
            self.name = name
            self.price = price
            self.quantity = quantity

        def add(self,quantity):
            self.quantity += quantity

        def remove(self,quantity):
            self.quantity -= quantity

        def update(self,name,price,quantity):
            self.name = name
            self.price = price
            self.quantity = quantity

i1 = inventory("vi",9999,10)
i1.add(10)
print(i1.quantity)
i1.remove(5)
print(i1.quantity)
```

20
15

07) Create a Class with instance attributes of your choice.

```
In [ ]: import math

class Circle :
    def __init__(self,r) :
        self.r = r

    def area(self):
        return math.pi * self.r * self.r
    def perimeter(self):
        return 2 * math.pi * self.r

c1 = Circle(7)
c1.area()
c1.perimeter()
```

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

```
In [28]: class StudentKit:
    principal = "jay"
    principal2 = "Mohit"

    def __init__(self,name):
        self.name = name
        self.attendance = 0

    def atten(self,days):
        self.attendance += (days/365) * 100

    def certificate(self):

        if(self.attendance > 75):
            print("You Are Certified.")
        else:
            print("Attandance Is Below")

s1 = StudentKit("vi")
s1.atten(290)
s1.certificate()
```

You Are Certified.

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

```
In [37]: class Time:
    def __init__(self,h,m,s):
        self.h = h
        self.m = m
        self.s = s

    def add(self,t2):
        total_h = self.h + t2.h
        total_m = self.m + t2.m
        total_s = self.s + t2.s

        if(total_s >= 60):
            total_m += total_s // 60
            total_s = total_s % 60

        if(total_m >= 60):
            total_h += total_m // 60
```

```
        total_m = total_m % 60

        total_h = total_h % 12

    return Time(total_h,total_m,total_s)

t1 = Time(1,20,50)
t2 = Time(1,50,50)
t3 = t1.add(t2)
print(f"{t3.h} : {t3.m} : {t3.s}")
```

3 : 11 : 40



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.: 459

Name: Vibhu Shihora

Continued..

10) Calculate area of a rectangle using object as an argument to a method.

```
In [7]: class Rectangle :  
  
        def area(self,side,side2):  
            return side * side2  
  
        r = Rectangle()  
  
        print(r.area(5,6))
```

30

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

```
In [4]: class Square :  
  
        def __init__(self,side):  
            self.side = side  
  
        def area(self):  
            return self.side * self.side  
  
        def output(self):
```

```

        return "Area : " + str(self.area())

r = Square(10)

print(r.output())
# print(r.side)

```

Area : 100
10

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named `area()` and a method named `output()` that prints the output and is invoked by `area()`.

Also define a class method that compares the two sides of rectangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : **THIS IS SQUARE**.

```

In [12]: class ItsNotRectangle(Exception):
        pass

        class Rectangle :

            def __init__(self,side,side2):
                self.side , self.side2 = self.compareSide(side,side2)

            def area(self):
                return self.side * self.side2

            def output(self):
                return "Area : " + str(self.area())

            @classmethod
            def compareSide(cls, side1, side2):
                if side1 == side2:
                    raise ItsNotRectangle("THIS IS SQUARE")
                return side1, side2

        try:
            r = Rectangle(10,20)
        except Exception as msg:
            print(msg)
        else:
            print(r.output())

```

Area : 200

13) Define a class Square having a private attribute "side".

Implement `get_side` and `set_side` methods to access the private attribute from outside of the class.

```
In [7]: class Square :

    def __init__(self,side):
        self.__side = side

    def area(self):
        return self.__side * self.__side

    def setSide(self,newSide):
        self.__side = newSide

    def getSide(self):
        return self.__side

    def output(self):
        return "Area : " + str(self.area())

r = Square(10)

# print(r.output())

# print(r.getSide()) # Get

r.setSide(20) # Set

print(r.getSide()) # Get

# print(r.__side) #We Cannot Access Private Member Directly
```

20

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balance. It has two methods getBalance() and printBalance().

```
In [12]: class Profit:

    def getProfit(self):
        self.profit = float(input("Enter Profit: "))

class Loss:

    def getLoss(self):
        self.loss = float(input("Enter Loss: "))

class BalanceSheet(Profit, Loss):

    def getBalance(self):
        return self.profit - self.loss

    def printBalance(self):
```

```

        print("Balance Is : " + str(self.getBalance()))

b = BalanceSheet()
b.getProfit()
b.getLoss()
b.getBalance()
b.printBalance()

```

Balance Is : 79.0

15) WAP to demonstrate all types of inheritance.

In [14]: *# 1.Single Level*

```

class Parent:
    def show(self):
        print("This is Parent class")

class Child(Parent):
    def display(self):
        print("This is Child class")

```

```

c = Child()
c.show()
c.display()

```

2.Multiple

```

class Father:
    def showFather(self):
        print("Father class")

class Mother:
    def showMother(self):
        print("Mother class")

```

```

class Child(Father, Mother):
    def showChild(self):
        print("Child class")

```

```

c = Child()
c.showFather()
c.showMother()
c.showChild()

```

3.Multilevel

```

class Grandparent:
    def gp(self):
        print("Grandparent class")

```

```

class Parent(Grandparent):
    def p(self):

```

```
        print("Parent class")

class Child(Parent):
    def c(self):
        print("Child class")

obj = Child()
obj.gp()
obj.p()
obj.c()
```

4.Hierarchical

```
class Parent:
    def show(self):
        print("Parent class")

class Child1(Parent):
    def c1(self):
        print("Child1 class")

class Child2(Parent):
    def c2(self):
        print("Child2 class")

obj1 = Child1()
obj2 = Child2()

obj1.show()
obj1.c1()

obj2.show()
obj2.c2()
```

5.Hybrid

```
class A:
    def methodA(self):
        print("Class A")

class B(A):
    def methodB(self):
        print("Class B")

class C(A):
    def methodC(self):
        print("Class C")

class D(B, C):
    def methodD(self):
        print("Class D")
```

```

obj = D()
obj.methodA()
obj.methodB()
obj.methodC()
obj.methodD()

# ama java ni jem if apde child no constructor banvi e to parent no constructor
# super().__init__() child na constructor ma

# Yes, in Python you can call super().__init__() inside any method
# In Java, you cannot call a constructor from a normal method

# class Parent:
#     def __init__(self):
#         print("Parent constructor")

# class Child(Parent):
#     pass

# c = Child()

# ama If child does NOT define constructor, then parent is called automatically

```

This is Parent class

This is Child class

Father class

Mother class

Child class

Grandparent class

Parent class

Child class

Parent class

Child1 class

Parent class

Child2 class

Class A

Class B

Class C

Class D

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the `init` method in Employee to call the parent class's `init` method using the `super()` and then initialize the salary attribute.

```

In [18]: class Person:

         def __init__(self, name, age):
             self.name = name

```

```

        self.age = age

class Employee(Person):

    def __init__(self,name,age,salary):
        super().__init__(name,age)
        self.salary = salary

    def __str__(self):
        return "Name : " + self.name + "\nAge : " + str(self.age) + "\nSalary :

e = Employee("vi",19,99999999)
print(e)

```

```

Name : vi
Age : 19
Salary : 99999999

```

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

In [20]: `from abc import ABC, abstractmethod`

```

# ABC etle Abstract Base Class

```

```

class Shape(ABC):

    @abstractmethod
    def draw(self):
        pass

class Rectangle(Shape):
    def draw(self):
        print("Rectangle")

class Circle(Shape):
    def draw(self):
        print("Circle")

class Triangle(Shape):
    def draw(self):
        print("Triangle")

s = []
s.append(Rectangle())
s.append(Circle())
s.append(Triangle())

```

```
for i in s:  
    i.draw()
```

Rectangle

Circle

Triangle